Zellic

Prepared for
pablo@inverter.network
marving@inverter.network
Inverter Network

Prepared by
Katerina Belotskaia
Quentin Lemauf
Zellic

November 25, 2025

iTRY
Smart Contract Security Assessment

Contents

About Zellic **4**

1. Overview **4**

1.1. Executive Summary 5

1.2. Goals of the Assessment 5

1.3. Non-goals and Limitations 5

1.4. Results 5

2. Introduction **6**

2.1. About iTRY 7

2.2. Methodology 7

2.3. Scope 9

2.4. Project Overview 10

2.5. Project Timeline 11

3. Detailed Findings **11**

3.1. A blacklisted user can transfer tokens on behalf of non-blacklisted users using allowance 12

3.2. The `MIN_SHARES` threshold can lock shares below this minimum amount 13

3.3. The `redistributeLockedAmount` does not validate that the resulting `totalSupply` is not less than the minimum threshold 15

3.4. iTry backing can fall below 1:1 on NAV drop 17

3.5. Native fee loss on failed `wiTryVaultComposer.LzReceive` execution 18

3.6. Nonstandard ERC-20 tokens may break the transfer function 20

DRAFT

4. Threat Model **21**

4.1. Component: Cross-chain interaction 22

4.2. Module: FastAccessVault.sol 28

4.3. Module: StakediTryCooldown.sol 30

4.4. Module: StakediTryCrosschain.sol 35

4.5. Module: StakediTryFastRedeem.sol 44

4.6. Module: StakediTry.sol 49

4.7. Module: UnstakeMessenger.sol 53

4.8. Module: YieldForwarder.sol 54

4.9. Module: iTryIssuer.sol 56

4.10. Module: iTrySilo.sol 61

4.11. Module: iTryTokenOFT.sol 62

4.12. Module: iTry.sol 71

4.13. Module: wiTryOFT.sol 79

4.14. Module: wiTryVaultComposer.sol 84

5. Assessment Results **92**

5.1. Disclaimer 93

DRAFT

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the #1 CTF (competitive hacking) team worldwide in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io and follow @zellic_io on Twitter. If you are interested in partnering with Zellic, contact us at hello@zellic.io.

DRAFT

1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for Inverter Network from November 18th to November 25th, 2025. During this engagement, Zellic reviewed iTRY's code for security vulnerabilities, design issues, and general weaknesses in security posture.

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Is unbacked minting of iTry possible?
- Is there a risk of theft or loss of funds during staking/unstaking, especially during cross-chain operations?
- Are there any blacklist/whitelist bugs?

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- Oracle component
- The unchanged part of the OrderManager contract
- Front-end components
- Infrastructure relating to the project
- Key custody

Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

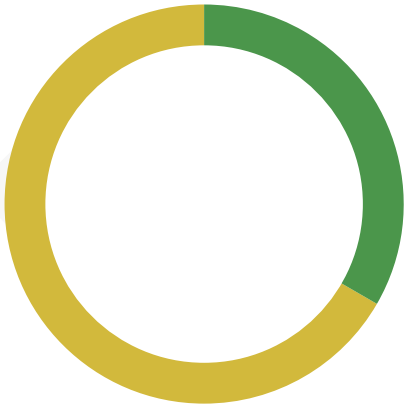
1.4. Results

During our assessment on the scoped iTRY contracts, we discovered six findings. No critical issues were found. Four findings were of medium impact and two were of low impact.

DRAFT

Breakdown of Finding Impacts

Impact Level	Count
Critical	0
High	0
Medium	4
Low	2
Informational	0



DRAFT

2. Introduction

2.1. About iTRY

Inverter Network contributed the following description of iTRY:

> The iTRY tokenization protocol enables the minting and redemption of turkish-lira-pegged iTRY tokens (and their staked, yield-bearing counterpart wiTRY), which are backed by Digital Liquidity Fund (DLF) tokens which represent shares of a traditional fund investing in Turkish Money Market Funds (MMF). The protocol includes cross-chain functionality via LayerZero integration.

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Basic coding mistakes. Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. Depending on the engagement, we may also employ sophisticated analyzers such as model checkers, theorem provers, fuzzers, and so on as necessary. We also perform a cursory review of the code to familiarize ourselves with the contracts.

Business logic errors. Business logic is the heart of any smart contract application. We examine the specifications and designs for inconsistencies, flaws, and weaknesses that create opportunities for abuse. For example, these include problems like unrealistic tokenomics or dangerous arbitrage opportunities. To the best of our abilities, time permitting, we also review the contract logic to ensure that the code implements the expected functionality as specified in the platform's design documents.

Integration risks. Several well-known exploits have not been the result of any bug within the contract itself; rather, they are an unintended consequence of the contract's interaction with the broader DeFi ecosystem. Time permitting, we review external interactions and summarize the associated risks: for example, flash loan attacks, oracle price manipulation, MEV/sandwich attacks, and so on.

Code maturity. We look for potential improvements in the codebase in general. We look for violations of industry best practices and guidelines and code quality standards. We also provide suggestions for possible optimizations, such as gas optimization, upgradability weaknesses, centralization risks, and so on.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no

hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood. We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

DRAFT

2.3. Scope

The engagement involved a review of the following targets:

iTRY Contracts

Type	Solidity
Platform	EVM-compatible
Target	iTry-contracts
Repository	https://github.com/InverterNetwork/iTry-contracts ↗
Version	47a82ca2bec004d55254d5d1920f86da9cf9d9ec
Programs	iTryIssuer.sol FastAccessVault.sol YieldForwarder.sol StakediTryFastRedeem.sol StakediTryCrosschain.sol wiTryVaultComposer.sol UnstakeMessenger.sol iTryTokenOFT.sol wiTryTokenOFT.sol StakediTry.sol iTry.sol iTryTokenOFTAdapter.sol StakediTryCooldown.sol wiTryOFTAdapter.sol

DRAFT

Target	Only changes between 1f6719f...da14d9d
Repository	https://github.com/InverterNetwork/iTRY-monorepo ↗
Version	`da14d9d355225990d7b35efb06bdb75db6da76ff`
Programs	`OrderManager.sol`

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of 1.8 person-weeks. The assessment was conducted by two consultants over the course of 1.2 calendar weeks.

DRAFT

Contact Information

The following project manager was associated with the engagement:

Pedro Moura
Engagement Manager
pedro@zellic.io

The following consultants were engaged to conduct the assessment:

Katerina Belotskaia
Engineer
kate@zellic.io

Quentin Lemauf
Engineer
quentin@zellic.io

2.5. Project Timeline

The key dates of the engagement are detailed below.

November 18, 2025	Kick-off call
November 18, 2025	Start of primary review period
November 25, 2025	End of primary review period

DRAFT

3. Detailed Findings

3.1. A blacklisted user can transfer tokens on behalf of non-blacklisted users using allowance

Target	wiTryOFT		
Category	Business Logic	**Severity**	Medium
Likelihood	Medium	**Impact**	Medium

Description

The wiTryOFT contract is a wrapper around the OFT contract from LayerZero that represents the wiTry token on spoke chains. Besides the cross-chain–transfer functionality, it also supports standard ERC-20 functions for same-chain transfers.

The `_beforeTokenTransfer` function validates only the `from` and `to` addresses to ensure they are not blacklisted during token transfer. However, the ERC-20 `transferFrom` function allows any caller to move tokens on behalf of another user when the caller has been granted sufficient allowance. Because `_beforeTokenTransfer` does not validate `msg.sender`, a blacklisted caller can still initiate a same-chain token transfer on behalf of a non-blacklisted user as long as allowance exists.

This behavior does not apply to cross-chain transfers, since cross-chain sending always uses `msg.sender` as the initiator and cannot rely on allowance.

Impact

A blacklisted caller is able to initiate transferring for other users using allowance, bypassing the intended blacklist restrictions.

Recommendations

We recommend to update the `_beforeTokenTransfer` function to validate the `msg.sender` address, ensuring that blacklisted addresses cannot initiate `transferFrom` operations even when they possess allowance.

Remediation

This issue has been acknowledged by Inverter Network, and a fix was implemented in commit `a93afe29` ↗.

DRAFT

3.2. The MIN_SHARES threshold can lock shares below this minimum amount

Target	StakediTry		
Category	Business Logic	**Severity**	Medium
Likelihood	Medium	**Impact**	Medium

Description

StakediTry enforces a fixed minimum total share supply (MIN_SHARES) after every deposit or withdrawal:

```
function _checkMinShares() internal view {
    uint256 _totalSupply = totalSupply();
    if (_totalSupply > 0 && _totalSupply < MIN_SHARES)
    revert MinSharesViolation();
}
```

In the case where the remaining total share supply is close to MIN_SHARES, any further withdrawal that reduces totalSupply into the interval (0, MIN_SHARES) will revert due to MinSharesViolation.

In such cases, a small remaining amount of shares (and corresponding underlying assets) may remain stuck in the vault, because no single holder can perform a withdrawal that leaves totalSupply either exactly 0 or at least MIN_SHARES.

Impact

Amounts of shares that would bring totalSupply into the interval (0, MIN_SHARES) may become non-withdrawable and remain locked in the vault.

Recommendations

Consider adapting the OZ virtual shares pattern ↗ or creating dead shares during deployment or the first deposit to keep totalSupply safely above the MIN_SHARES threshold.

Remediation

This issue has been acknowledged by Inverter Network.

The team acknowledges the behavior related to the `MIN_SHARES` threshold and has decided not to modify the ERC-4626 implementation. Instead, they plan to address the issue operationally by creating “dead” shares at deployment time.

DRAFT

3.3. The `redistributeLockedAmount` does not validate that the resulting `totalSupply` is not less than the minimum threshold

Target	StakediTry		
Category	Business Logic	**Severity**	Medium
Likelihood	Medium	**Impact**	Medium

Description

The StakediTry contract allows users to stake iTry tokens and receive in exchange wiTry tokens. Additionally, this contract supports the `SOFT_RESTRICTED_STAKER_ROLE` and `FULL_RESTRICTED_STAKER_ROLE` modes for the staker accounts, which impose restrictions on the transfer, staking, or unstaking of funds. And also, for every `deposit` or `withdraw` action, this contract validates that the `totalSupply` has not fallen below the minimum number of shares, not including zero.

The `redistributeLockedAmount` allows the admin of the contract to transfer the staking balance of the fully restricted account with `FULL_RESTRICTED_STAKER_ROLE` mode. The initial balance of the staker will be burned and can be minted again for an unrestricted account address, or staked iTry tokens will be placed as a reward. In the second case, the wiTry tokens are not minted, so the total amount of shares will be decreased, but the `_checkMinShares` validation is not performed in this case.

```
function redistributeLockedAmount(address from, address to)
    external nonReentrant onlyRole(DEFAULT_ADMIN_ROLE) {
    if (hasRole(FULL_RESTRICTED_STAKER_ROLE, from) &&
    !hasRole(FULL_RESTRICTED_STAKER_ROLE, to)) {
        uint256 amountToDistribute = balanceOf(from);
        uint256 iTryToVest = previewRedeem(amountToDistribute);
        _burn(from, amountToDistribute);
        // to address of address(0) enables burning
        if (to == address(0)) {
            _updateVestingAmount(iTryToVest);
        } else {
            _mint(to, amountToDistribute);
        }

        emit LockedAmountRedistributed(from, to, amountToDistribute);
    } else {
        revert OperationNotAllowed();
    }
}
```

Impact

As a result of executing the `redistributeLockedAmount` function, the `totalSupply` amount may fall within a prohibited range between `0` and the `MIN_SHARES` amount. Subsequent legitimate deposits or withdrawals operations that do not increase `totalSupply` to the `MIN_SHARES` value will be blocked.

Recommendations

We recommend adding a check to the `redistributeLockedAmount` function to ensure that the `totalSupply` value does not fall within the prohibited range.

Remediation

This issue has been acknowledged by Inverter Network, and a fix was implemented in commit `e1cd2396` ↗.

DRAFT

3.4. iTry backing can fall below 1:1 on NAV drop

Target	iTryIssuer		
Category	Business Logic	**Severity**	Medium
Likelihood	Low	**Impact**	Medium

Description

The iTry token is intended to stay fully backed 1:1 by DLF deposits, assuming the MMF NAV only increases. If `navPrice` ever falls below 1 (MMF drawdown), existing iTry supply remains unchanged while the DLF backing shrinks, leaving iTry partially unbacked and no on-chain path to reestablish the invariant:

- The stated mitigation is to pause issuance and "wait until the underlying MMF recovers", but neither is enforced on chain.
- A permissioned burn that reduces `_totalIssuedITry` without paying out DLF was mentioned as a future idea, but it is not currently implemented. Without it, there is no on-chain mechanism to retire iTry or inject collateral to restore 1:1 if NAV drops.
- Pausing alone prevents further dilution but does not resolve the existing shortfall; iTry holders remain exposed to an undercollateralized state.

Impact

If NAV drops below 1, iTry becomes undercollateralized with no guaranteed, on-chain remediation. Holders bear insolvency risk until a top-up or discretionary admin intervention occurs.

Recommendations

Implement the proposed permissioned burn (admin-controlled) that can reduce outstanding iTry yield supply without DLF payout.

Remediation

This issue has been acknowledged by Inverter Network, and a fix was implemented in commit `af3ada39` ↗.

3.5. Native fee loss on failed `wiTryVaultComposer.LzReceive` execution

Target	wiTryVaultComposer		
Category	Business Logic	**Severity**	Low
Likelihood	Low	**Impact**	Low

Description

The `unstake` function allows the caller to initiate a cross-chain unstaking process. Since successfully unstaked funds will be transferred back to the spoke chain, the caller should specify and provide the sufficient `returnTripAllocation` value to cover the fee for the transferring of the cross-chain message to the hub chain and back to the spoke chain.

```
function unstake(uint256 returnTripAllocation)
    external payable nonReentrant returns (bytes32 guid) {
    // [...]
    bytes memory callerOptions =
        OptionsBuilder.newOptions().addExecutorLzReceiveOption(LZ_RECEIVE_GAS,
    uint128(returnTripAllocation));
    bytes memory options = _combineOptions(hubEid, MSG_TYPE_UNSTAKE,
    callerOptions);

    // Quote with native drop included (single quote with fixed
    returnTripAllocation)
    MessagingFee memory fee = _quote(hubEid, payload, options, false);
    // [...]
    // Automatic refund to msg.sender
    MessagingReceipt memory receipt = _lzSend(
        hubEid,
        payload,
        options,
        fee,
        payable(msg.sender) // Refund excess to user
    );
    // [...]
    }
```

Because the `returnTripAllocation` value is estimated up front, two negative scenarios are possible:

1. **Overpayment.** During `wiTryVaultComposer.LzReceive` execution, the specified fee will

be provided to the `endpoint.send` function, which will refund all unspent native fees to the specified `_refundAddress` address. Since it is not possible to perform a direct refund to the user on the other chain, these funds will be transferred to the wiTryVaultComposer contract itself. This contract supports the `rescueToken` function, which allows the owner to transfer tokens, including native tokens, from the contract to an arbitrary address, after which the owner can manually process the funds as intended. But a self-claim by the user is not supported.

2. **Underpayment.** The fee provided was insufficient, and as a result, the `lzReceive` function on the hub side will revert. Even if, in this case, the unstake message remains stored at the LZ endpoint contract and can be retried by any caller by supplying sufficient fees, the initially paid fee will remain with the LayerZero Executor balance, and no functionality is implied to refund the unspent fee. As a result, users must pay the full fee again to successfully complete the cross-chain transfer of unstaked funds.

Impact

In the case of underpayment, users will lose their fee and will have to pay twice to complete the unstake request.

Recommendations

Consider storing all failed cross-chain messages (provided to the `lzReceive` function) inside the wiTryVaultComposer contract instead of relying on the LayerZero endpoint (where failed messages are kept for redelivery). The native fee provided for message delivery should also be stored and associated with the specific message, so it can be reused when the message is retried.

Remediation

This issue has been acknowledged by Inverter Network. Inverter Network has provided the following response:

> The team acknowledges the issue and discussed refund strategies with the LayerZero team during the integration phase. According to LayerZero's guidance, attempting to refund the user's fee on-chain would consume nearly all available gas and is therefore not recommended.
>
> As a result, the team does not plan to introduce a protocol-level refund mechanism, following LayerZero's guidance. Instead, they will focus on ensuring that the frontend estimates LayerZero fees as accurately as possible to minimize over- or under-funding. The team acknowledges and accepts the remaining risk of non-refundable native fees in edge cases.

3.6. Nonstandard ERC-20 tokens may break the transfer function

Target	Multiple Contracts		
Category	Business Logic	**Severity**	Low
Likelihood	Low	**Impact**	Low

Description

Some functions in the current protocol use `SafeERC20.safeTransfer`, while others use the direct `transfer` call. When the contracts interact only with known tokens that are expected to follow the ERC-20 standard, the `transfer` function can be used. However, for example, rescue functionality may need to handle any token that was mistakenly sent to the contract, and there is no guarantee that such tokens strictly follow the ERC-20 standard.

```
function rescueToken(address token, address to, uint256 amount)
    external onlyOwner nonReentrant {
    if (to == address(0)) revert CommonErrors.ZeroAddress();
    if (amount == 0) revert CommonErrors.ZeroAmount();

    if (token == address(0)) {
        // Rescue ETH
        (bool success,) = to.call{value: amount}("");
        if (!success) revert CommonErrors.TransferFailed();
    } else {
        // Rescue ERC20 tokens
        if (!IERC20(token).transfer(to, amount)) {
            revert CommonErrors.TransferFailed();
        }
    }

    emit TokensRescued(token, to, amount);
}
```

Impact

If a nonstandard token is recovered using a raw `transfer`, the function may appear to succeed even though no tokens were transferred, or it may revert unexpectedly. This can result in tokens becoming stuck in the contract, which breaks the token-rescue mechanism.

Recommendations

Use `SafeERC20.safeTransfer` in all rescue functions, since these functions may interact with arbitrary tokens. For the main protocol flow, using `transfer` is acceptable only if the team is certain that all involved tokens strictly follow the ERC-20 standard. For consistency and safety, adopting `SafeERC20` everywhere is also an option.

Remediation

This issue has been acknowledged by Inverter Network, and a fix was implemented in commit `31184f2a` ↗.

DRAFT

4. Threat Model

This provides a full threat model description for various functions. As time permitted, we analyzed each function in the contracts and created a written threat model for some critical functions. A threat model documents a given function's externally controllable inputs and how an attacker could leverage each input to cause harm.

Not all functions in the audit scope may have been modeled. The absence of a threat model in this section does not necessarily suggest that a function is safe.

For the `iTry` and `iTryTokenOFT` sections, note that their logic is forked from audited and tested Ethena code which explain why there is no explicit tests associated. Nevertheless a best practice would be to add and maintain test suites whenever the codebase is modified to avoid discrepancies with the upstream fork.

4.1. Component: Cross-chain interaction

Overview

The protocol uses a hub-and-spoke topology, where the hub chain is Ethereum Mainnet and the spoke chain is `megaETH`.

There are two tokens: 1) iTry (hub native) and iTryTokenOFT (spoke representation) and 2) wiTry (hub wrapped) and wiTryOFT (spoke representation). On the hub chain, the adapter contracts iTryTokenOFTAdapter and wiTryOFTAdapter lock the underlying tokens (iTry and wiTry, respectively).

Users can transfer tokens between hub and spoke chains (standard OFT transfers). They can also interact with a vault via cross-chain messages using the compose functionality (`INITIATE_COOLDOWN`, `FAST_REDEEM`) as well as deposit iTry and receive vault share tokens.

Vault-related cross-chain logic is mediated by wiTryVaultComposer, UnstakeMessenger, and StakediTryCrosschain (vault implementation).

Components

In this section we discuss the hub and spoke chains.

Hub chain

There are several components in the hub chain.

There is **iTryTokenOFTAdapter**, which holds locked iTry tokens corresponding to the iTryTokenOFT supply on spoke, allows users to initiate the cross-chain sending of iTry to the spoke, and releases iTry for the receiver when tokens are transferred back from the spoke.

There is **wiTryOFTAdapter**, which holds locked wiTry corresponding to wiTryOFT supply on spoke, allows users to initiate the cross-chain sending of wiTry to the spoke, and releases wiTry for the receiver when tokens are transferred back from the spoke.

An entry point for cross-chain vault operations (`INITIATE_COOLDOWN`, `FAST_REDEEM`, and asset deposits), **wiTryVaultComposer** does the following:

- Receives OFT assets and share tokens from the spoke chain and processes `composeMsg` using the `lzCompose` function to handle received funds
- Calls into StakediTryCrosschain to perform vault actions
- Supports OApp functionality to process the inbound cross-chain messages using `lzReceive` for the unstake request case
- Implements a refund mechanism for failing operations

The ERC-4626 vault contract **StakediTryCrosschain** as an addition to the vault functionality, manages cross-chain staking, cooldown, unstaking, and fast redemption. Here are the supported functions for interaction with the wiTryVaultComposer:

- `cooldownSharesByComposer` (used for `INITIATE_COOLDOWN`)
- `cooldownAssetsByComposer` (unused as of the time of writing)
- `unstakeThroughComposer` (used by `UnstakeMessenger.unstake`)
- `fastRedeemThroughComposer` (used for `FAST_REDEEM`)
- `fastWithdrawThroughComposer` (unused as of the time of writing)

Spoke chain

There are several components in the spoke chain.

There is **iTryTokenOFT**, which is a representation of the iTry OFT token on the spoke chain. It mints/burns tokens in response to cross-chain messages.

There is **wiTryOFT**, which is a representation of wiTry OFT on the spoke chain. It mints/burns tokens in response to cross-chain messages.

There is also **UnstakeMessenger**, which allows a user on the spoke to finalize the unstake process after cooldown via the `unstake` function. The unstake request will be sent cross-chain and processed by the wiTryVaultComposer contract.

Flows

Here we give an overview of the standard token-transfer flows, the asynchronous vault flows, the `INITIATE_COOLDOWN` flow, and the `FAST_REDEEM` flow.

Standard token-transfer flows

This is the cross-chain transfer process of the wiTry token from the hub chain to the spoke chain.

1. A user calls the `iTryTokenOFTAdapter.send()` function where they specify the amount to be transferred, the receiver address, the `dstEid`, the fee (native tokens or LZ token),

extraOptions, and composeMsg. The composeMsg can be arbitrary in this case, and the receiver of the tokens in the dstEid should handle it by themselves. The transferred amount of tokens will be transferred from the caller to the iTryTokenOFTAdapter contract and will be locked there until receiving tokens from the other chain.

2. Then, the endpoint.send() function performs fee validation, distributes the fee, and emits packet information for DVNs, executors, and other actors.
3. On the dstEid side, the delivered message payload is saved in the endpoint contract for further execution.
4. The executor performs the endpoint.lzReceive() function call to finalize token transferring.
5. The wiTryOFT.lzReceive() function is executed to mint the delivered amount of tokens for the receiver.
6. If composeMsg has been provided, it will be saved using the endpoint.sendCompose() function call in the endpoint contract for the receiver address. After that, anyone can trigger endpoint.lzCompose() to execute the saved composeMsg, if the recipient supports this functionality.

The flow is symmetric for transferring from the hub chain to the spoke chain.

1. A user initiates sending of wiTryOFT on the spoke.
2. wiTryOFT burns the sent amount on the spoke.
3. The endpoint sends a message to the hub.
4. On the hub, wiTryOFTAdapter.lzReceive() is called.
5. The adapter releases the corresponding wiTry from its balance to the receiver.

For iTry/iTryTokenOFT, the process is analogous: tokens are burned on spoke and unlocked from iTryTokenOFTAdapter on the hub.

Asynchronous vault flows via wiTryVaultComposer

In addition to simple transfers, users on the spoke chain can initiate vault operations on the hub chain via cross-chain messages. These operations use the OFT compose mechanism and a command flag oftCmd.

There are two commands supported by the wiTryVaultComposer contract on the hub chain:

1. oftCmd == INITIATE_COOLDOWN
2. oftCmd == FAST_REDEEM

Additionally, users can send iTry tokens to wiTryVaultComposer to deposit into the vault and receive share tokens in the spoke chain.

The `INITIATE_COOLDOWN` flow

There is also the `INITIATE_COOLDOWN` flow. This process allows a user to initiate an unstaking cooldown and receive the iTry tokens back on the spoke chain later.

On the hub chain, a user sends wiTry tokens to wiTryVaultComposer on the hub with `oftCmd = bytes("INITIATE_COOLDOWN")` and `composeMsg` encoded appropriately (see below).

```
SendParam memory compose_params = SendParam({
    dstEid: 0,                // Not used for _initiateCooldown
    to: address(0),           // Not used for _initiateCooldown
    amountLD: 0,              // Not used for _initiateCooldown
    minAmountLD: 0,           // Not used for _initiateCooldown
    extraOptions: "",         // Not used for _initiateCooldown
    composeMsg: "",           // Not used for _initiateCooldown
    oftCmd: bytes("INITIATE_COOLDOWN")
});
```

On this hub chain, the following occurs.

1. The wiTryOFTAdapter receives tokens and transfers them to the wiTryVaultComposer contract. The `composeMsg` is saved in the endpoint contract for the wiTryVaultComposer.
2. The `wiTryVaultComposer.lzCompose()` function is executed with `composeMsg` and executes the process depending on the decoding in the `composeMsg` command.
3. In this case, the `INITIATE_COOLDOWN` triggers `StakediTryCrosschain.cooldownSharesByComposer` function execution.
4. The wiTryVaultComposer provides the specified shares-token amount to perform the unstake operation. The released assets are transferred to the silo contract until finalizing the whole process.
5. A user's unstaked funds are locked in silo and can be received after cooldown.

After `INITIATE_COOLDOWN` and once the cooldown has passed, a user should transfer a cross-chain unstake message using the `UnstakeMessenger.unstake` function to finalize the unstaking. To correctly perform this action, the user should execute the `wiTryVaultComposer.quoteUnstakeReturn(user, amount, spokeDstEid)` on the hub chain in advance to get the necessary fee amount for sending unstaking tokens back to the spoke chain. After that, the `UnstakeMessenger.unstake` function with the full necessary fee can be executed to transfer the unstake request.

If the `INITIATE_COOLDOWN` request has not been executed on behalf of the user previously, the `IStakediTryCrosschain(address(VAULT)).unstakeThroughComposer(user);` function will revert since the user does not have any active unstake cooldowns. Also, users cannot initiate the unstake process on behalf of other users or specify an arbitrary address to unstake and steal funds,

since the `UnstakeMessenger.unstake` uses only the `msg.sender` address as a user address. During the `unstakeThroughComposer` execution, the current cooldown request is reset to zero to prevent double unstaking.

There are some **safety notes** to consider. The `UnstakeMessenger.unstake` uses `msg.sender` as the user address, so other users cannot initiate unstake on behalf of someone else or specify an arbitrary unstake address and steal funds. Additionally, incorrect fee handling during `unstake` execution can lead to losing the fee. See more details in Finding 3.5. ↗.

The `FAST_REDEEM` flow

This operation allows redeeming vault assets quickly using share tokens (and the fee will be charged in this case). This flow occurs as follows:

1. On the hub side in the StakediTryCrosschain contract, the fast-redeem mode should be enabled.
2. The user must supply share tokens to the hub for fast redeem.
3. Tokens previously sent for `INITIATE_COOLDOWN` cannot be reused for `FAST_REDEEM`.
4. The user pays a fee for the fast-redeem operation.

After sending the `FAST_REDEEM` request from the spoke chain, these steps will be performed on the hub:

1. First, wiTryVaultComposer receives share tokens and executes the `FAST_REDEEM` command.
2. It calls `StakediTryCrosschain.fastRedeemThroughComposer`.
3. The user pays a fee and withdraws the rest of the funds.
4. Cross-chain sending is initiated to transfer assets to the specified recipient address on the `_dstEid` chain.

```
SendParam memory compose_params = SendParam({
    dstEid: _dstEid,            // ID of destination chain (spoke) supported by
    the system
    to: address(recipient),     // Recipient of tokens on dstEid
    amountLD: 0,                // Arbitrary; overridden by `assets` amount
    minAmountLD: 0,             // Arbitrary; overridden by `assets` amount
    extraOptions: "",           // Extra options per LayerZero docs
    composeMsg: "",             // Optional application-level data
    oftCmd: bytes("FAST_REDEEM")
});
```

For all these commands, the compose data should be encoded along with `minMsgValue`:

```
bytes memory composeMsg = abi.encode(compose_params, minMsgValue);
```

The `minMsgValue` specifies the minimum `msg.value` required when `lzCompose` is executed on wiTryVaultComposer. It covers the case where `lzCompose` itself needs to initiate cross-chain transfers and must pay LayerZero fees (e.g., for deposit and `FAST_REDEEM` flows). For the `INITIATE_COOLDOWN` case, it can be specified as zero.

Deposits

Users can send iTry tokens from the spoke to the wiTryVaultComposer contract on the hub; the specific `oftCmd` is not necessary in this case. However, `composeMsg` should contain the `SendParam` for the recipient address and `dstEid`.

The wiTryVaultComposer contract will deposit these tokens into StakediTryCrosschain and transfer those returned from the vault share amount of tokens back to the user on the spoke chain.

Invariants

Below are the invariants for **iTryTokenOFTAdapter** and **wiTryOFTAdapter**.

- The total amount of wiTry locked in the wiTryOFTAdapter is equal to the wiTryOFT token amount in spoke chains.
- The total amount of iTry locked in the iTryTokenOFTAdapter is equal to the iTryTokenOFT token amount in spoke chains.
- A user can only finalize unstake if the `INITIATE_COOLDOWN` operation has been executed before and the cooldown has passed. These funds cannot be redeemed a different way.
- A user can only unstake their own tokens and cannot specify any other address to unstake.
- The `INITIATE_COOLDOWN` operation is executed only if the user transferred the necessary amount of share tokens to the wiTryVaultComposer in the hub chain.
- Users cannot control the amount of asset tokens that they receive in exchange for the provided share tokens (the assets returned are determined solely by vault logic and current vault state).
- For `FAST_REDEEM`, a user should send share tokens to the wiTryVaultComposer. Previously transferred funds for `INITIATE_COOLDOWN` cannot be redeemed.

Attack surface

We recommend implementing the following controls to reduce the cross-chain attack surface:

1. It should be ensured that `unstakeThroughComposer` cannot be called unless cooldown exists and has completed.
2. The `UnstakeMessenger.unstake` should not allow specifying an arbitrary target user.
3. The cooldown record is cleared on a successful `unstakeThroughComposer`, which is protected against message replay on `MSG_TYPE_UNSTAKE`.
4. Incorrect fees can lead to failed cross-chain sending, stuck funds, or unprocessed operations.
5. Since any user can save `composeMsg` in the endpoint contract for an arbitrary receiver, the `lzCompose` handlers must validate the sender address of the `composeMsg`, and only messages from the trusted sender should be processed. Also, only expected and trusted tokens should be processed in this function.
6. The `lzCompose` should never use the amount value specified by the user. Only the amount decoded to the cross-chain message can be trusted, and it reflects the real amount of tokens received. Also, the `composeFrom` address from the composed message is not controlled by the user, so `lzCompose` can use this address as a trusted user address who initiated this cross-chain transferring. However, the validation from point 5 should be performed to guarantee that an arbitrary `composeMsg` cannot be executed.

4.2. Module: FastAccessVault.sol

Function: `processTransfer(address _receiver, uint256 _amount)`

This function is called by the issuer when DLF is redeemed for iTry. It is only callable by the iTryIssuer contract.

Inputs

- `_receiver`
  - **Control**: Issuer access control.
  - **Constraints**: N/A.
  - **Impact**: Address receiving DLF tokens from the vault.
- `_amount`
  - **Control**: Issuer access control.
  - **Constraints**: N/A.
  - **Impact**: Amount of DLF tokens sent from the vault to the recipient.

Branches and code coverage

Intended branches

- The vault possesses enough DLF tokens.
  - ☑ Test coverage
- The `_amount` of DLF tokens are sent to the receiver.
  - ☑ Test coverage

Negative behavior

- The caller is the issuer or address(0).
  - ☑ Negative test
- The vault has enough balance to cover the transfer.
  - ☑ Negative test
- The token return fals on transfer.
  - ☑ Negative test

Function call analysis

- `this._vaultToken.balanceOf(address(this))`
  - **What is controllable?** N/A.
  - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
  - **What happens if it reverts, reenters or does other unusual control flow?** N/A.
- `this._vaultToken.transfer(_receiver, _amount)`
  - **What is controllable?** N/A.
  - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
  - **What happens if it reverts, reenters or does other unusual control flow?** N/A.

Function: `rebalanceFunds()`

This function rebalances the vault's DLF funds with a custodian address base on a buffer limit. If the vault balance is under the buffer limit, it asks for a top-up via the event `TopUpRequestedFromCustodian`. If the vault balance is above the buffer limit, it sends the difference to the custodian address.

Branches and code coverage

Intended branches

- Fetch the `targetBalance` and `currentBalance`.
  - ☑ Test coverage
- Top-up or transfer the DLT rebalance amount if necessary.
  - ☑ Test coverage

Function call analysis

- `this._issuerContract.getCollateralUnderCustody()`
  - **What is controllable?** N/A.
  - **If the return value is controllable, how is it used and how can it go wrong?** Return the total amount of DLF collateral held under custody.
  - **What happens if it reverts, reenters or does other unusual control flow?** N/A.
- `this._vaultToken.balanceOf(address(this))`
  - **What is controllable?** N/A.
  - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
  - **What happens if it reverts, reenters or does other unusual control flow?** N/A.
- `this._vaultToken.transfer(this.custodian, excess)`
  - **What is controllable?** `custodian` is set while deploying this contract or by an owner-restricted function; `excess` is determined by the buffer diff balance.
  - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
  - **What happens if it reverts, reenters or does other unusual control flow?** N/A.

4.3. Module: StakediTryCooldown.sol

Function: `cooldownAssets(uint256 assets)`

This function starts a cooldown period for the sender with `assets` amount of tokens. Anyone can start a cooldown period to unstake at the end of this precise period. It is only callable when cooldown mode is on.

Inputs

- `assets`
  - **Control**: Amount of assets to unstake after the cooldown period.

- **Constraints**: The balance of the sender.
- **Impact**: Place these tokens in the silo until the period ends and the user unstakes.

Branches and code coverage

Intended branches

- The sender possesses enough assets in the vault.
  - ☑ Test coverage
- Populate `cooldowns` with the new period end timestamp and the asset amount.
  - ☑ Test coverage

Negative behavior

- The call is done while cooldown mode is off.
  - ☑ Negative test
- A new cooldown period starts over an existing one.
  - ☐ Negative test

Function call analysis

- `this.maxWithdraw(msg.sender) -> this._convertToAssets(this.balanceOf(owner), Rounding.Down) -> Math.mulDiv(shares, this.totalAssets() + 1, this.totalSupply() + 10 ** this._decimalsOffset(), rounding)`
  - **What is controllable?** N/A.
  - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
  - **What happens if it reverts, reenters or does other unusual control flow?** N/A.
- `this.maxWithdraw(msg.sender) -> this._convertToAssets(this.balanceOf(owner), Rounding.Down) -> this.totalAssets() -> this._asset.balanceOf(address(this))`
  - **What is controllable?** N/A.
  - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
  - **What happens if it reverts, reenters or does other unusual control flow?** N/A.
- `this.maxWithdraw(msg.sender) -> this._convertToAssets(this.balanceOf(owner), Rounding.Down) ->`

`this.totalAssets() -> IERC20(this.asset()).balanceOf(address(this))`

- **What is controllable?** N/A.
- **If the return value is controllable, how is it used and how can it go wrong?** Returns the sender balance.
- **What happens if it reverts, reenters or does other unusual control flow?** N/A.

- `this.previewWithdraw(assets) -> this._convertToShares(assets, Rounding.Up) -> Math.mulDiv(assets, this.totalSupply() + 10 ** this._decimalsOffset(), this.totalAssets() + 1, rounding)`
  - **What is controllable?** N/A.
  - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
  - **What happens if it reverts, reenters or does other unusual control flow?** N/A.
- `this._withdraw(msg.sender, address(this.silo), msg.sender, assets, shares) -> super._withdraw(caller, receiver, _owner, assets, shares) -> SafeERC20.safeTransfer(this._asset, receiver, assets)`
  - **What is controllable?** N/A.
  - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
  - **What happens if it reverts, reenters or does other unusual control flow?** N/A.

Function: `cooldownShares(uint256 shares)`

This function starts a cooldown period for the sender with `shares` amount. Anyone can start a cooldown period to unstake at the end of this precise period. It is only callable when the cooldown mode is on.

Inputs

- `shares`
  - **Control**: Amount of shares converted in assets to unstake after the cooldown period.
  - **Constraints**: Balance of the sender.
  - **Impact**: Places these converted shares in the silo until the period ends and the user unstakes.

DRAFT

Branches and code coverage

Intended branches

- The sender possesses enough shares in the vault.
  - ☑ Test coverage
- Populate `cooldowns` with the new period end timestamp and the asset amount.
  - ☑ Test coverage

Negative behavior

- The call is done while cooldown mode is off.
  - ☑ Negative test
- A new cooldown period starts over an existing one.
  - ☐ Negative test

Function call analysis

- `this.previewRedeem(shares) -> this._convertToAssets(shares, Rounding.Down) -> Math.mulDiv(shares, this.totalAssets() + 1, this.totalSupply() + 10 ** this._decimalsOffset(), rounding)`
  - **What is controllable?** N/A.
  - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
  - **What happens if it reverts, reenters or does other unusual control flow?** N/A.
- `this.previewRedeem(shares) -> this._convertToAssets(shares, Rounding.Down) -> this.totalAssets() -> this._asset.balanceOf(address(this))`
  - **What is controllable?** N/A.
  - **If the return value is controllable, how is it used and how can it go wrong?** Returns the sender balance.
  - **What happens if it reverts, reenters or does other unusual control flow?** N/A.
- `this.previewRedeem(shares) -> this._convertToAssets(shares, Rounding.Down) -> this.totalAssets() -> IERC20(this.asset()).balanceOf(address(this))`
  - **What is controllable?** N/A.
  - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
  - **What happens if it reverts, reenters or does other unusual control flow?**

N/A.

- `this._withdraw(msg.sender, address(this.silo), msg.sender, assets, shares) -> super._withdraw(caller, receiver, _owner, assets, shares) -> SafeERC20.safeTransfer(this._asset, receiver, assets)`
    - **What is controllable?** N/A.
    - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
    - **What happens if it reverts, reenters or does other unusual control flow?** N/A.

Function: `unstake(address receiver)`

This function unstakes tokens from the silo when the redeemer cooldown period ends. Only the redeemer can call the unstake function and choose the receiver. It is only possible to unstake funds when the cooldown period is fully finished.

Inputs

- `receiver`
    - **Control**: Anyone can call this function.
    - **Constraints**: N/A.
    - **Impact**: Receive the funds from the silo.

Branches and code coverage

Intended branches

- `msg.sender` possesses a nonzero `userCooldown`.
    - ☑ Test coverage
- The `userCooldown` mapping is reset, and funds are sent from the silo to `receiver`.
    - ☑ Test coverage

Negative behavior

- The sender cooldown period is not yet finished.
    - ☑ Negative test

Function call analysis

- `this.silo.withdraw(receiver, assets)`

- **What is controllable?** `receiver` is caller-controlled; `assets` is determined while starting the cooldown period.
- **If the return value is controllable, how is it used and how can it go wrong?** N/A.
- **What happens if it reverts, reenters or does other unusual control flow?** N/A.

4.4. Module: StakediTryCrosschain.sol

Function: `cooldownAssetsByComposer(uint256 assets, address redeemer)`

The composer specifies an asset amount to lock for a redeemer, converts to shares, burns them from the composer, and records a cooldown for the redeemer.

Inputs

- `assets`
  - **Control**: Composer caller.
  - **Constraints**: Must be `<= maxWithdraw(composer)` and nonzero (enforced by `_withdraw`).
  - **Impact**: Determines shares burned and assets placed into cooldown.
- `redeemer`
  - **Control**: Composer provided.
  - **Constraints**: Nonzero address.
  - **Impact**: Receives the cooldown entitlement for the specified assets.

Branches and code coverage

Intended branches

- Burns composer shares and transfers the assets into the silo on behalf of the redeemer address.
  - ☑ Test coverage

Negative behavior

- The caller is not the composer.
  - ☑ Negative test

DRAFT

Function call analysis

- `this.maxWithdraw(composer) -> this._convertToAssets(this.balanceOf(owner), Rounding.Down) -> Math.mulDiv(shares, this.totalAssets() + 1, this.totalSupply() + 10 ** this._decimalsOffset(), rounding)`
    - **What is controllable?** N/A.
    - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
    - **What happens if it reverts, reenters or does other unusual control flow?** N/A.
- `this.maxWithdraw(composer) -> this._convertToAssets(this.balanceOf(owner), Rounding.Down) -> this.totalAssets() -> this._asset.balanceOf(address(this))`
    - **What is controllable?** N/A.
    - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
    - **What happens if it reverts, reenters or does other unusual control flow?** N/A.
- `this.maxWithdraw(composer) -> this._convertToAssets(this.balanceOf(owner), Rounding.Down) -> this.totalAssets() -> IERC20(this.asset()).balanceOf(address(this))`
    - **What is controllable?** N/A.
    - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
    - **What happens if it reverts, reenters or does other unusual control flow?** N/A.
- `this.previewWithdraw(assets) -> this._convertToShares(assets, Rounding.Up) -> Math.mulDiv(assets, this.totalSupply() + 10 ** this._decimalsOffset(), this.totalAssets() + 1, rounding)`
    - **What is controllable?** N/A.
    - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
    - **What happens if it reverts, reenters or does other unusual control flow?** N/A.
- `this._startComposerCooldown(composer, redeemer, shares, assets) -> this._withdraw(composer, address(this.silo), composer, assets, shares) -> super._withdraw(caller, receiver, _owner, assets, shares) -> SafeERC20.safeTransfer(this._asset, receiver, assets)`
    - **What is controllable?** N/A.
    - **If the return value is controllable, how is it used and how can it go wrong?** N/A.

DRAFT

- **What happens if it reverts, reenters or does other unusual control flow?** N/A.

Function: `cooldownSharesByComposer(uint256 shares, address redeemer)`

The composer burns its own `shares` to start a cooldown on behalf of `redeemer`, locking the converted assets in the silo and returning the locked asset amount.

Inputs

- `shares`
  - **Control**: The composer provided with `COMPOSER_ROLE`.
  - **Constraints**: Must be `<= maxRedeem(composer)` and nonzero due to `_withdraw`.
  - **Impact**: Determines how many shares are burned from the composer and how many assets enter cooldown for the redeemer.
- `redeemer`
  - **Control**: The composer provided with `COMPOSER_ROLE`.
  - **Constraints**: Must be a nonzero address.
  - **Impact**: The cooldown entry is recorded for this address, enabling unstaking later.

Branches and code coverage

Intended branches

- Burn composer shares and transfer the assets into the silo on behalf of the redeemer address.
  - ☑ Test coverage

Negative behavior

- The caller is not the composer.
  - ☑ Negative test

Function call analysis

- `this.previewRedeem(shares) -> this._convertToAssets(shares, Rounding.Down) -> Math.mulDiv(shares, this.totalAssets() + 1, this.totalSupply() + 10 ** this._decimalsOffset(), rounding)`

    - **What is controllable?** N/A.
    - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
    - **What happens if it reverts, reenters or does other unusual control flow?** N/A.
- `this.previewRedeem(shares) -> this._convertToAssets(shares, Rounding.Down) -> this.totalAssets() -> this._asset.balanceOf(address(this))`
    - **What is controllable?** N/A.
    - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
    - **What happens if it reverts, reenters or does other unusual control flow?** N/A.
- `this.previewRedeem(shares) -> this._convertToAssets(shares, Rounding.Down) -> this.totalAssets() -> IERC20(this.asset()).balanceOf(address(this))`
    - **What is controllable?** N/A.
    - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
    - **What happens if it reverts, reenters or does other unusual control flow?** N/A.
- `this._startComposerCooldown(composer, redeemer, shares, assets) -> this._withdraw(composer, address(this.silo), composer, assets, shares) -> super._withdraw(caller, receiver, _owner, assets, shares) -> SafeERC20.safeTransfer(this._asset, receiver, assets)`
    - **What is controllable?** N/A.
    - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
    - **What happens if it reverts, reenters or does other unusual control flow?** N/A.

Function: `fastRedeemThroughComposer(uint256 shares, address crosschainReceiver, address owner)`

The composer bypasses cooldown by redeeming `shares` with a fast-redeem fee; it receives net assets locally for bridging. Shares are previously sent to the composer via the cross-chain OFT transfer or a local shares transfer.

Inputs

- `shares`

        - **Control**: Composer caller.
        - **Constraints**: Must be inferior to `maxRedeem(composer)`.
        - **Impact**: Determines the amount of assets sent back to the composer.
    - `crosschainReceiver`
        - **Control**: Composer caller.
        - **Constraints**: Must be nonzero.
        - **Impact**: N/A.
    - `owner`
        - **Control**: Composer caller.
        - **Constraints**: N/A.
        - **Impact**: N/A.

Branches and code coverage

Intended branches

- Burn shares, compute the corresponding assets, and send `netAmount` to the composer, while fees are sent to the treasury.
    - ☑ Test coverage

Negative behavior

- The caller is `COMPOSER_ROLE`.
    - ☑ Negative test

Function call analysis

- `this.previewRedeem(shares) -> this._convertToAssets(shares, Rounding.Down) -> Math.mulDiv(shares, this.totalAssets() + 1, this.totalSupply() + 10 ** this._decimalsOffset(), rounding)`
    - **What is controllable?** N/A.
    - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
    - **What happens if it reverts, reenters or does other unusual control flow?** N/A.
- `this.previewRedeem(shares) -> this._convertToAssets(shares, Rounding.Down) -> this.totalAssets() -> this._asset.balanceOf(address(this))`
    - **What is controllable?** N/A.
    - **If the return value is controllable, how is it used and how can it go wrong?**

N/A.
    - **What happens if it reverts, reenters or does other unusual control flow?** N/A.
- `this.previewRedeem(shares) -> this._convertToAssets(shares, Rounding.Down) -> this.totalAssets() -> IERC20(this.asset()).balanceOf(address(this))`
    - **What is controllable?** N/A.
    - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
    - **What happens if it reverts, reenters or does other unusual control flow?** N/A.
- `this._redeemWithFee(shares, totalAssets, composer, composer) -> this.previewWithdraw(feeAssets) -> this._convertToShares(assets, Rounding.Up) -> Math.mulDiv(assets, this.totalSupply() + 10 ** this._decimalsOffset(), this.totalAssets() + 1, rounding)`
    - **What is controllable?** N/A.
    - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
    - **What happens if it reverts, reenters or does other unusual control flow?** N/A.

Function: `fastWithdrawThroughComposer(uint256 assets, address crosschainReceiver, address owner)`

The composer requests an immediate withdrawal of `assets` during cooldown, pays the fast-redeem fee, and receives the net assets locally for bridging.

Inputs

- `assets`
    - **Control**: Composer caller.
    - **Constraints**: Must be inferior to `maxWithdraw(composer)`.
    - **Impact**: Determines shares burned and fee/net asset split.
- `crosschainReceiver`
    - **Control**: Composer caller.
    - **Constraints**: Nonzero address.
    - **Impact**: N/A.
- `owner`
    - **Control**: Composer caller.

- **Constraints**: N/A.
- **Impact**: N/A.

Branches and code coverage

Intended branches

- Burn shares, compute the corresponding assets, and send `netAmount` to the composer, while fees are sent to the treasury.

  ☑ Test coverage

Negative behavior

- The caller is `COMPOSER_ROLE`.

  ☑ Negative test

Function call analysis

- `this.maxWithdraw(composer) -> this._convertToAssets(this.balanceOf(owner), Rounding.Down) -> Math.mulDiv(shares, this.totalAssets() + 1, this.totalSupply() + 10 ** this._decimalsOffset(), rounding)`
  - **What is controllable?** N/A.
  - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
  - **What happens if it reverts, reenters or does other unusual control flow?** N/A.
- `this.maxWithdraw(composer) -> this._convertToAssets(this.balanceOf(owner), Rounding.Down) -> this.totalAssets() -> this._asset.balanceOf(address(this))`
  - **What is controllable?** N/A.
  - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
  - **What happens if it reverts, reenters or does other unusual control flow?** N/A.
- `this.maxWithdraw(composer) -> this._convertToAssets(this.balanceOf(owner), Rounding.Down) -> this.totalAssets() -> IERC20(this.asset()).balanceOf(address(this))`
  - **What is controllable?** N/A.
  - **If the return value is controllable, how is it used and how can it go wrong?** N/A.

DRAFT

    - **What happens if it reverts, reenters or does other unusual control flow?** N/A.
- `this.previewWithdraw(assets) -> this._convertToShares(assets, Rounding.Up) -> Math.mulDiv(assets, this.totalSupply() + 10 ** this._decimalsOffset(), this.totalAssets() + 1, rounding)`
    - **What is controllable?** N/A.
    - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
    - **What happens if it reverts, reenters or does other unusual control flow?** N/A.
- `this._redeemWithFee(shares, assets, composer, composer) -> this._withdraw(this._msgSender(), this.fastRedeemTreasury, owner, feeAssets, feeShares) -> super._withdraw(caller, receiver, _owner, assets, shares) -> SafeERC20.safeTransfer(this._asset, receiver, assets)`
    - **What is controllable?** N/A.
    - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
    - **What happens if it reverts, reenters or does other unusual control flow?** N/A.

Function: `unstakeThroughComposer(address receiver)`

The composer completes a finished cooldown for `receiver`, clears their cooldown record, and pulls the cooled assets from the silo back to the composer.

Inputs

- `receiver`
    - **Control**: The composer-provided redeemer whose cooldown is consumed.
    - **Constraints**: Nonzero address.
    - **Impact**: The associated cooldown entry is reset, and assets are withdrawn to the composer for cross-chain delivery.

Branches and code coverage

Intended branches

- Fetch the `receiver` cooldown's values; if the cooldown period is finished, transfer the tokens from the silo to `receiver`.
    - ☑ Test coverage

Negative behavior

- The caller is not `COMPOSER_ROLE`.
  - ☑ Negative test

Function call analysis

- `this.silo.withdraw(msg.sender, assets)`
  - **What is controllable?** `assets` is determined by the amount previously cooled down.
  - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
  - **What happens if it reverts, reenters or does other unusual control flow?** N/A.

Function: `_startComposerCooldown(address composer, address redeemer, uint256 shares, uint256 assets)`

This is an internal helper that withdraws assets from the composer to the silo and records a cooldown for the `redeemer`.

Inputs

- `composer`
  - **Control**: Provided by the caller (`cooldown*ByComposer`).
  - **Constraints**: Must have sufficient shares/allowance and not be restricted.
  - **Impact**: Shares are burned, and assets are sent from this address to the silo.
- `redeemer`
  - **Control**: Provided by the caller.
  - **Constraints**: Assumed validated before call (nonzero).
  - **Impact**: Receives the cooldown allocation; their cooldown struct is updated.
- `shares`
  - **Control**: Provided by the caller.
  - **Constraints**: Must be nonzero and match the assets conversion, or `_withdraw` reverts.
  - **Impact**: Burned from `composer` during `_withdraw`.
- `assets`
  - **Control**: Derived from previews in the caller.
  - **Constraints**: Must align with `shares` — nonzero enforced by `_withdraw`.

DRAFT

- **Impact**: The amount transferred to the silo and recorded in the redeemer's cooldown.

Branches and code coverage

Intended branches

- compute the new cooldown timestamp, withdraw assets to the silo, update the mapping state with the new values.
  - ☐ Test coverage

4.5. Module: StakediTryFastRedeem.sol

Function: `fastRedeem(uint256 shares, address receiver, address owner)`

This function allows a user to bypass the cooldown by redeeming `shares` immediately, charging a fast-redeem fee that is sent to the treasury and transferring the net assets to `receiver`.

Inputs

- `shares`
  - **Control**: The caller must be `owner` or have allowance for `owner`'s shares.
  - **Constraints**: Must be inferior to `maxRedeem(owner)` and large enough that the computed fee is nonzero.
  - **Impact**: Determines how many shares are burned from `owner` — and used to define the converted assets amount.
- `receiver`
  - **Control**: Caller-specified.
  - **Constraints**: Must not hold `FULL_RESTRICTED_STAKER_ROLE`.
  - **Impact**: Receives the net assets (`totalAssets - fee`).
- `owner`
  - **Control**: Caller-specified shareholder.
  - **Constraints**: Must own/approve sufficient shares and must not have the `FULL_RESTRICTED_STAKER_ROLE`.
  - **Impact**: Shares and allowances are consumed.

Branches and code coverage

Intended branches

- The owner possesses enough shares.
  - ☑ Test coverage
- The computed fees are not zero.
  - ☑ Test coverage
- Net funds are transferred to `receiver`, and fees are transferred to `fastRedeemTreasury`.
  - ☑ Test coverage

Negative behavior

- Cooldown mode is on, and the `fastRedeem` functionality is allowed.
  - ☑ Negative test

Function call analysis

- `this.previewRedeem(shares) -> this._convertToAssets(shares, Rounding.Down) -> Math.mulDiv(shares, this.totalAssets() + 1, this.totalSupply() + 10 ** this._decimalsOffset(), rounding)`
  - **What is controllable?** The caller controls `shares`, `totalAssets` / `totalSupply` depend on vault state and can be shifted via deposits/withdrawals.
  - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
  - **What happens if it reverts, reenters or does other unusual control flow?** N/A.
- `this.previewRedeem(shares) -> this._convertToAssets(shares, Rounding.Down) -> this.totalAssets() -> this._asset.balanceOf(address(this))`
  - **What is controllable?** N/A.
  - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
  - **What happens if it reverts, reenters or does other unusual control flow?** N/A.
- `this.previewRedeem(shares) -> this._convertToAssets(shares, Rounding.Down) -> this.totalAssets() -> IERC20(this.asset()).balanceOf(address(this))`
  - **What is controllable?** N/A.
  - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
  - **What happens if it reverts, reenters or does other unusual control flow?** N/A.

DRAFT

- `this._redeemWithFee(shares, totalAssets, receiver, owner) -> this.previewWithdraw(feeAssets) -> this._convertToShares(assets, Rounding.Up) -> Math.mulDiv(assets, this.totalSupply() + 10 ** this._decimalsOffset(), this.totalAssets() + 1, rounding)`
    - **What is controllable?** N/A.
    - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
    - **What happens if it reverts, reenters or does other unusual control flow?** N/A.

Function: `fastWithdraw(uint256 assets, address receiver, address owner)`

This function withdraws a specific asset amount immediately during cooldown by charging a fast-redeem fee to the treasury and sending the net assets to `receiver`.

Inputs

- `assets`
    - **Control**: The caller must be `owner` or have allowance from `owner`.
    - **Constraints**: Must be inferior to `maxWithdraw(owner)` and large enough that the calculated fee is nonzero.
    - **Impact**: Determines how many shares are burned and the fee/net asset amounts transferred.
- `receiver`
    - **Control**: Caller-specified.
    - **Constraints**: Must not have the `FULL_RESTRICTED_STAKER_ROLE`.
    - **Impact**: Recipient of the net assets.
- `owner`
    - **Control**: Caller-specified.
    - **Constraints**: Must hold/approve sufficient shares and not have the `FULL_RESTRICTED_STAKER_ROLE`.
    - **Impact**: Receive the assets in exchange for fees.

Branches and code coverage

Intended branches

- The owner possesses enough assets.
    - ☑ Test coverage
- Computed fees are not zero.

- ☑ Test coverage
- Net funds are transferred to `receiver`; fees are transferred to `fastRedeemTreasury`.
  - ☑ Test coverage

Negative behavior

- Cooldown mode is on, and the `fastRedeem` functionality is allowed.
  - ☑ Negative test

Function call analysis

- `this.maxWithdraw(owner) -> this._convertToAssets(this.balanceOf(owner), Rounding.Down) -> Math.mulDiv(shares, this.totalAssets() + 1, this.totalSupply() + 10 ** this._decimalsOffset(), rounding)`
  - **What is controllable?** N/A.
  - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
  - **What happens if it reverts, reenters or does other unusual control flow?** N/A.
- `this.maxWithdraw(owner) -> this._convertToAssets(this.balanceOf(owner), Rounding.Down) -> this.totalAssets() -> this._asset.balanceOf(address(this))`
  - **What is controllable?** N/A.
  - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
  - **What happens if it reverts, reenters or does other unusual control flow?** N/A.
- `this.maxWithdraw(owner) -> this._convertToAssets(this.balanceOf(owner), Rounding.Down) -> this.totalAssets() -> IERC20(this.asset()).balanceOf(address(this))`
  - **What is controllable?** N/A.
  - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
  - **What happens if it reverts, reenters or does other unusual control flow?** N/A.
- `this.previewWithdraw(assets) -> this._convertToShares(assets, Rounding.Up) -> Math.mulDiv(assets, this.totalSupply() + 10 ** this._decimalsOffset(), this.totalAssets() + 1, rounding)`
  - **What is controllable?** N/A.
  - **If the return value is controllable, how is it used and how can it go wrong?** N/A.

DRAFT

- **What happens if it reverts, reenters or does other unusual control flow?** N/A.

Function: `_redeemWithFee(uint256 shares, uint256 assets, address receiver, address owner)`

This function splits a fast redemption into a treasury fee and a net payout by computing the fee, ensuring it is nonzero, and then performing two `_withdraw` calls (treasury and receiver).

Inputs

- `shares`
  - **Control**: Provided by the calling function (`fastRedeem` / `fastWithdraw`) based on user input or assets conversion.
  - **Constraints**: Must be sufficient to cover `feeShares` and `netShares`.
  - **Impact**: The total amount of shares burned from `owner`.
- `assets`
  - **Control**: Provided by the calling function (`fastRedeem` / `fastWithdraw`) based on user input or shares conversion.
  - **Constraints**: Must produce a nonzero fee.
  - **Impact**: Determines `feeAssets`, `netAssets`, and the corresponding shares burned.
- `receiver`
  - **Control**: Provided by the external caller.
  - **Constraints**: Must not hold the `FULL_RESTRICTED_STAKER_ROLE`.
  - **Impact**: Receives the assets after fee deduction.
- `owner`
  - **Control**: Provided by the external caller.
  - **Constraints**: Must have enough shares/allowance and not hold the `FULL_RESTRICTED_STAKER_ROLE`.
  - **Impact**: Pays both the treasury fee and net redemption via burned shares.

Branches and code coverage

Intended branches

- Computed fees are not zero.
  - ☑ Test coverage
- Burn shares, and transfer asset to the receiver and `fastRedeemTreasury`.

DRAFT

- ☑ Test coverage

4.6. Module: StakediTry.sol

Function: `addToBlacklist(address target, bool isFullBlacklisting)`

This function blacklists an address from interacting with the share vault. It cannot blacklist the owner, and the function is only callable by the `BLACK_LIST_MANAGER`.

Note that the address cannot revoke the blacklist role because of the function override.

Inputs

- `target`
  - **Control**: Access control by `BLACKLIST_MANAGER_ROLE`.
  - **Constraints**: N/A.
  - **Impact**: The address will not be able to interact with the vault shares.
- `isFullBlacklisting`
  - **Control**: Access control by `BLACKLIST_MANAGER_ROLE`.
  - **Constraints**: N/A.
  - **Impact**: `FULL` blocks any shares movement, while `SOFT` only blocks the deposit functionality.

Branches and code coverage

Intended branches

- Blacklist role is assigned to `target`.
  - ☐ Test coverage

Function: `getUnvestedAmount()`

This function computes the tokens left to unvest for the current vesting period and returns `0` if the vesting period ends.

Branches and code coverage

Intended branches

- Compute the ratio of tokens left to unlock for the time left in the current vesting period.

- ☐ Test coverage

Function: `redistributeLockedAmount(address from, address to)`

This function redistributes the funds of a `FULL_RESTRICTED_STAKER_ROLE` to another address not blacklisted — access control by `DEFAULT_ADMIN_ROLE`.

If the recipient is `address(0)`, it burns the shares and creates a new vesting period; otherwise, it mints the tokens to the recipient address.

Inputs

- `from`
  - **Control**: Access control by `DEFAULT_ADMIN_ROLE`.
  - **Constraints**: Should be a blacklisted address.
  - **Impact**: Burns the shares of this precise address.
- `to`
  - **Control**: Access control by `DEFAULT_ADMIN_ROLE`.
  - **Constraints**: Should not be a blacklisted address.
  - **Impact**: If different than `address(0)`, mints tokens.

Branches and code coverage

Intended branches

- `from` is fully blacklisted; `to` is not fully blacklisted.
  - ☐ Test coverage
- Burn `from` shares, and start a vesting period or mint new tokens.
  - ☐ Test coverage

Negative behavior

- The caller is not `DEFAULT_ADMIN_ROLE`.
  - ☐ Negative test

Function call analysis

- `this.previewRedeem(amountToDistribute) -> this._convertToAssets(shares, Rounding.Down) -> Math.mulDiv(shares, this.totalAssets() + 1, this.totalSupply() + 10 ** this._decimalsOffset(), rounding)`

    - **What is controllable?** N/A.
    - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
    - **What happens if it reverts, reenters or does other unusual control flow?** N/A.
  - `this.previewRedeem(amountToDistribute) -> this._convertToAssets(shares, Rounding.Down) -> this.totalAssets() -> this._asset.balanceOf(address(this))`
    - **What is controllable?** N/A.
    - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
    - **What happens if it reverts, reenters or does other unusual control flow?** N/A.
  - `this.previewRedeem(amountToDistribute) -> this._convertToAssets(shares, Rounding.Down) -> this.totalAssets() -> IERC20(this.asset()).balanceOf(address(this))`
    - **What is controllable?** N/A.
    - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
    - **What happens if it reverts, reenters or does other unusual control flow?** N/A.

Function: `removeFromBlacklist(address target, bool isFullBlacklisting)`

This function removes the blacklist role from an address previously blacklisted. The function is only callable by the `BLACK_LIST_MANAGER`.

Inputs

- `target`
  - **Control**: Access control by `BLACKLIST_MANAGER_ROLE`.
  - **Constraints**: N/A.
  - **Impact**: The address will be able to interact with the vault shares.
- `isFullBlacklisting`
  - **Control**: Access control by `BLACKLIST_MANAGER_ROLE`.
  - **Constraints**: N/A.
  - **Impact**: Determine which type of blacklist role should be removed.

Branches and code coverage

Intended branches

- The target address role is removed.
  - ☐ Test coverage

Function: `transferInRewards(uint256 amount)`

This function is called by `yieldRecipient` to inflate share value with the new yield. Only `REWARDER_ROLE` can start a new vesting period with this new yield.

Inputs

- `amount`
  - **Control**: Access control by `REWARDER_ROLE`.
  - **Constraints**: N/A.
  - **Impact**: Start a new vesting period with `amount`.

Branches and code coverage

Intended branches

- Start a new vesting period.
  - ☑ Test coverage
- Transfer the iTry tokens into the vault.
  - ☑ Test coverage

Negative behavior

- The caller is a service admin.
  - ☐ Negative test
- A vesting period already exists for the current timestamp.
  - ☐ Negative test

Function call analysis

- `SafeERC20.safeTransferFrom(IERC20(this.asset()), msg.sender, address(this), amount)`
  - **What is controllable?** `amount` is caller-controlled.

- **If the return value is controllable, how is it used and how can it go wrong?** N/A.
- **What happens if it reverts, reenters or does other unusual control flow?** N/A.

4.7. Module: UnstakeMessenger.sol

Function: `unstake(uint256 returnTripAllocation)`

This function allows the caller to initiate a cross-chain unstaking process.

Inputs

- `returnTripAllocation`
  - **Control**: Full control.
  - **Constraints**: Should not be zero.
  - **Impact**: Determines the native fee value to cover the return part of the unstaking operation from the hub chain to the spoke chain. The user should get the `returnTripAllocation` amount from the `wiTryVaultComposer.quoteUnstakeReturn()` function, which correctly specifies user address, amount, and `dstEid` (the ID of the chain where this stake function is called).

Branches and code coverage

Intended branches

- The unstake message has been successfully sent to the hub chain.
  - ☑ Test coverage
- The `UnstakeRequested` event has been emitted.
  - ☑ Test coverage

Negative behavior

- The provided native amount is less than the required fee.
  - ☑ Negative test
- `returnTripAllocation` is zero.
  - ☑ Negative test

Function call analysis

- `this._quote(this.hubEid, payload, options, False) -> this.endpoint.quote(MessagingParams(_dstEid, this._getPeerOrRevert(_dstEid), _message, _options, _payInLzToken), address(this))`
    - **What is controllable?** N/A.
    - **If the return value is controllable, how is it used and how can it go wrong?** Returns the fee to perform the transfer of the unstake request to the hub chain. It should include the specified `returnTripAllocation` amount for the back trip. If the caller provided less native fee, the transaction will revert.
    - **What happens if it reverts, reenters or does other unusual control flow?** There are no problems here.
- `this._lzSend(this.hubEid, payload, options, fee, address payable(msg.sender)) -> this.endpoint.send{value: messageValue}(MessagingParams(_dstEid, this._getPeerOrRevert(_dstEid), _message, _options, _fee.lzTokenFee > 0), _refundAddress)`
    - **What is controllable?** N/A.
    - **If the return value is controllable, how is it used and how can it go wrong?** Returns the `MessagingReceipt` data. It is used to get the unique message ID to track the transferring process. In the case of successful execution, the caller address will get back the fee excess.
    - **What happens if it reverts, reenters or does other unusual control flow?** Reverts if the caller did not provide enough fee.

4.8. Module: YieldForwarder.sol

Function: `processNewYield(uint256 _newYieldAmount)`

This function is called while claiming the yield on `iTryIssuer::processAccumulatedYield`, which forwards the yield to the `yieldRecipient` address. Later, `yieldRecipient` will call `StakediTry::transferInRewards` to share the yield with iTry stakers.

Inputs

- `_newYieldAmount`
    - **Control**: Arbitrary caller, but in the case of yield forwarding from `iTryIssuer`, only `iTryIssuer::DEFAULT_ADMIN_ROLE` can execute the full flow.
    - **Constraints**: N/A.
    - **Impact**: Forward the yield from this contract to the `yieldRecipient` address.

Branches and code coverage

Intended branches

- Forward yield token to the `yieldRecipient` address.
  - ☑ Test coverage

Negative behavior

- `yieldRecipient` is `address(0)`.
  - ☑ Negative test
- `yieldToken` does not return a bool on failed transfer.
  - ☑ Negative test

Function call analysis

- `this.yieldToken.transfer(this.yieldRecipient, _newYieldAmount)`
  - **What is controllable?** The token amount transferred (`_newYieldAmount`) and the recipient are set while deploying the contract (`yieldRecipient`) or via `setYieldRecipient` (only callable by the owner).
  - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
  - **What happens if it reverts, reenters or does other unusual control flow?** N/A.

Function: `setYieldRecipient(address _newRecipient)`

This function changes the `yieldRecipient` address.

Inputs

- `_newRecipient`
  - **Control**: The owner.
  - **Constraints**: N/A
  - **Impact**: Changes the address receiving the `yieldTokens` in `processNewYield` or from the `ItryIssuer::processAccumulatedYield` flow.

Branches and code coverage

Intended branches

DRAFT

- The address is updated with the new one.
  - ☑ Test coverage

Negative behavior

- The new address is not zero.
  - ☑ Negative test
- The caller is not the owner.
  - ☑ Negative test

4.9. Module: iTryIssuer.sol

Function: `mintFor(address recipient, uint256 dlfAmount, uint256 minAmountOut)`

This function mints iTry tokens according to the Redstone `navPrice` and the DLF amount deposited into the contract. It is only callable by `_WHITELISTED_USER_ROLE`.

Inputs

- `recipient`
  - **Control**: `_WHITELISTED_USER_ROLE`-controlled.
  - **Constraints**: N/A.
  - **Impact**: Mints new iTry token to this address.
- `dlfAmount`
  - **Control**: `_WHITELISTED_USER_ROLE`-controlled.
  - **Constraints**: N/A.
  - **Impact**: Amount of inflow DLF tokens — mints iTry tokens by the `navPrice` and `dlfAmount`.
- `minAmountOut`
  - **Control**: `_WHITELISTED_USER_ROLE`-controlled.
  - **Constraints**: N/A.
  - **Impact**: Slippage protection against `navPrice` — reverts if the iTry token minted is less than `minAmountOut`.

Branches and code coverage

Intended branches

- Fetch the `navPrice` with the Redstone oracle.

- ☐ Test coverage
- Compute the number of iTry tokens to mint, and compare it to `minAmountOut`.
  - ☑ Test coverage
- `netDlfmount` is transferred in the vault.
  - ☑ Test coverage
- iTry tokens are minted to `recipient`.
  - ☑ Test coverage

Negative behavior

- The caller is not the owner.
  - ☑ Negative test

Function call analysis

- `this.oracle.price()`
  - **What is controllable?** N/A.
  - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
  - **What happens if it reverts, reenters or does other unusual control flow?** N/A.
- `this._transferIntoVault(msg.sender, netDlfAmount, feeAmount) -> this.collateralToken.transferFrom(from, address(this.liquidityVault), dlfAmount)`
  - **What is controllable?** `netDlfAmount` is derived from `dlfAmount`, which is caller-controlled; `liquidityVault` is deployed while deploying this contract and stored as immutable.
  - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
  - **What happens if it reverts, reenters or does other unusual control flow?** N/A.
- `this._transferIntoVault(msg.sender, netDlfAmount, feeAmount) -> this.collateralToken.transferFrom(from, this.treasury, feeAmount)`
  - **What is controllable?** `feeAmount` is derived from `dlfAmount` and `mintFeeInBPS`, which is caller-controlled; `liquidityVault` is deployed while deploying this contract and stored as immutable.
  - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
  - **What happens if it reverts, reenters or does other unusual control flow?** N/A.

DRAFT

- `this._mint(recipient, iTRYAmount) -> this.iTryToken.mint(receiver, amount)`
    - **What is controllable?** `amount` is derived from caller input and `navPrice`.
    - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
    - **What happens if it reverts, reenters or does other unusual control flow?** N/A.

Function: `processAccumulatedYield()`

Everyday this function recomputes the number of iTry tokens to be minted as yield based on the new `navPrice`.

Branches and code coverage

Intended branches

- Fetch `navPrice` through the oracle.
    - ☑ Test coverage
- If yield is available, mint the new iTry tokens to the `yieldReceiver` contract.
    - ☑ Test coverage
- Execute `processNewYield` on the `yieldReceiver` contract and transfer the yield to another address.
    - ☑ Test coverage

Negative behavior

- Caller is `DEFAULT_ADMIN_ROLE`.
    - ☑ Negative test

Function call analysis

- `this.oracle.price()`
    - **What is controllable?** N/A.
    - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
    - **What happens if it reverts, reenters or does other unusual control flow?** N/A.
- `this._mint(address(this.yieldReceiver), newYield) -> this.iTryToken.mint(receiver, amount)`

- **What is controllable?** `receiver` is owner-defined, and `amount` is derived from the new yield calculation via `navPrice` and `_totalDLFUnderCustody`.
- **If the return value is controllable, how is it used and how can it go wrong?** N/A.
- **What happens if it reverts, reenters or does other unusual control flow?** N/A.

- `this.yieldReceiver.processNewYield(newYield)`
  - **What is controllable?** `newYield` is defined by the `navPrice` increase and `_totalDLFUnderCustody`.
  - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
  - **What happens if it reverts, reenters or does other unusual control flow?** N/A.

Function: `redeemFor(address recipient, uint256 iTRYAmount, uint256 minAmountOut)`

This function redeems iTry tokens for DLF tokens, only callable by `_WHITELISTED_USER_ROLE`. It converts the iTry inflow token into DLF tokens with the Redstone `navPrice` and applies slippage with `minAmountOut`.

Inputs

- `recipient`
  - **Control**: `_WHITELISTED_USER_ROLE`-controlled.
  - **Constraints**: N/A.
  - **Impact**: Transfer DLF tokens to this address.
- `iTRYAmount`
  - **Control**: `_WHITELISTED_USER_ROLE`-controlled.
  - **Constraints**: N/A.
  - **Impact**: Amount of inflowing iTry tokens.
- `minAmountOut`
  - **Control**: `_WHITELISTED_USER_ROLE`-controlled.
  - **Constraints**: N/A.
  - **Impact**: Slippage protection against `navPrice` — revert if the DLF token amount is less than `minAmountOut`.

DRAFT

Branches and code coverage

Intended branches

- Fetch the `navPrice` with the Redstone oracle.
  - ☐ Test coverage
- Compute the number of DLF tokens to redeem, and compare it to `minAmountOut`.
  - ☑ Test coverage
- `iTryAmount` is burned.
  - ☑ Test coverage
- DLF tokens are transferred to the `recipient`.
  - ☑ Test coverage

Negative behavior

- The caller is not the owner.
  - ☑ Negative test

Function call analysis

- `this.oracle.price()`
  - **What is controllable?** N/A.
  - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
  - **What happens if it reverts, reenters or does other unusual control flow?** N/A.
- `this._burn(msg.sender, iTRYAmount) -> this.iTryToken.burnFrom(from, amount)`
  - **What is controllable?** `from` and `amount` are caller-controlled.
  - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
  - **What happens if it reverts, reenters or does other unusual control flow?** N/A.
- `this.liquidityVault.getAvailableBalance()`
  - **What is controllable?** N/A.
  - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
  - **What happens if it reverts, reenters or does other unusual control flow?** N/A.
- `this._redeemFromVault(recipient, netDlfAmount, feeAmount) ->`

DRAFT

`this.liquidityVault.processTransfer(receiver, receiveAmount)`

- **What is controllable?** `receiver` and `receiveAmount` are caller-controlled.
- **If the return value is controllable, how is it used and how can it go wrong?** N/A.
- **What happens if it reverts, reenters or does other unusual control flow?** N/A.

- `this._redeemFromVault(recipient, netDlfAmount, feeAmount) -> this.liquidityVault.processTransfer(this.treasury, feeAmount)`

  - **What is controllable?** `this.treasury` and `feeAmount` are owner-controlled and set before this function's execution.
  - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
  - **What happens if it reverts, reenters or does other unusual control flow?** N/A.

4.10. Module: iTrySilo.sol

Function: `withdraw(address to, uint256 amount)`

This function transfers iTry tokens from the silo to the `to` address; it is used to cool down assets while withdrawing or redeeming from the StakediTry vault.

Inputs

- `to`
  - **Control**: Only StakediTry-vault–callable.
  - **Constraints**: N/A.
  - **Impact**: Recipient of the iTry tokens.
- `amount`
  - **Control**: Only StakediTry-vault–callable.
  - **Constraints**: N/A.
  - **Impact**: Amount of iTry tokens sent.

Branches and code coverage

Intended branches

- Send iTry tokens to `to` with `amount` tokens.
  - ☑ Test coverage

Negative behavior

- Caller is not the staking vault.
  - ☑ Negative test

Function call analysis

- `this.iTry.transfer(to, amount)`
  - **What is controllable?** N/A.
  - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
  - **What happens if it reverts, reenters or does other unusual control flow?** N/A.

4.11. Module: iTryTokenOFT.sol

Function: `addBlacklistAddress(address[] users)`

This function allows the owner of the contract to blacklist addresses. Blacklisted addresses cannot transfer tokens and receive tokens. This function does not emit any events.

There is no test coverage for this function.

Inputs

- `users`
  - **Control**: Full control.
  - **Constraints**: N/A.
  - **Impact**: A blacklisted user cannot transfer or receive iTry tokens.

Branches and code coverage

Intended branches

- If a user is already blacklisted, they will be blacklisted again.
  - ☐ Test coverage
- If a user is already whitelisted, the whitelisted status will be revoked and the user will be blacklisted.
  - ☐ Test coverage
- If a user was not blacklisted/whitelisted before, they will become blacklisted.

DRAFT

- ☐ Test coverage

Negative behavior

- The caller is not an owner.
  - ☐ Negative test

Function: `addWhitelistAddress(address[] users)`

This function allows the owner of the contract to add users to the whitelist. If the mode is `TransferState.WHITELIST_ENABLED`, only the whitelisted users can transfer and receive tokens. There is no test coverage for this function. This function does not emit any events.

Inputs

- `users`
  - **Control**: Full control.
  - **Constraints**: N/A.
  - **Impact**: List of users' addresses that will be added to the whitelist.

Branches and code coverage

Intended branches

- If a user is already whitelisted, they will be whitelisted again.
  - ☐ Test coverage
- If a user is already blacklisted, the blacklisted status will be revoked and the user will be whitelisted.
  - ☐ Test coverage
- If the user was not blacklisted/whitelisted before, they will become whitelisted.
  - ☐ Test coverage

Negative behavior

- The caller is not an owner.
  - ☐ Negative test

Function: `redistributeLockedAmount(address from, address to)`

This function allows the owner of the contract to move the full balance of the blacklisted user to the other non-blacklisted account. Firstly, tokens are burned and after that minted again to the

non-blacklisted account.

Inputs

- `from`
  - **Control**: Full control.
  - **Constraints**: Should be a blacklisted account.
  - **Impact**: Tokens from this account will be burned.
- `to`
  - **Control**: Full control.
  - **Constraints**: Should be a non-blacklisted account.
  - **Impact**: The full balance of the blacklisted account will be minted for the non-blacklisted account.

Branches and code coverage

Intended branches

- The function is successfully executed when the mode is `TransferState.WHITELIST_ENABLED`.
  - ☐ Test coverage
- The function is successfully executed when the mode is `TransferState.FULLY_ENABLED`.
  - ☐ Test coverage
- The balance of the `from` account is zero.
  - ☐ Test coverage
- The balance of `to` has been increased by the initial `from` balance value.
  - ☐ Test coverage

Negative behavior

- The caller is not an owner.
  - ☐ Negative test
- The `from` address is not blacklisted.
  - ☐ Negative test
- The `to` address is blacklisted.
  - ☐ Negative test
- The `from` balance is zero.

- ☐ Negative test
- The mode is `TransferState.FULLY_DISABLED`.
  - ☐ Negative test

Function: `removeBlacklistAddress(address[] users)`

This function allows the owner of contract to remove an address of the users from the blacklist. This function does not emit any events.

There is no test coverage for this function.

Inputs

- `users`
  - **Control**: Full control.
  - **Constraints**: N/A.
  - **Impact**: Addresses of the users who will be removed from the blacklist.

Branches and code coverage

Intended branches

- A user address has been blacklisted and has been successfully removed from the blacklist.
  - ☐ Test coverage

Negative behavior

- A user address is not blacklisted initially.
  - ☐ Negative test
- The caller is not an owner.
  - ☐ Negative test

Function: `removeWhitelistAddress(address[] users)`

This function allows the owner of the contract to remove users from the whitelist. If the mode is `TransferState.WHITELIST_ENABLED`, only the whitelisted users can transfer and receive tokens. So the users removed from the whitelist cannot perform a transfer operation after removing from the whitelist. There is no test coverage for this function. This function does not emit any events.

Inputs

- `users`
  - **Control**: Full control.
  - **Constraints**: N/A.
  - **Impact**: The list of users' addresses that will be removed from the whitelist.

Branches and code coverage

Intended branches

- A user address has been whitelisted and has been successfully removed from the whitelist.
  - ☐ Test coverage

Negative behavior

- The caller is not an owner.
  - ☐ Negative test

Function: `setMinter(address _newMinter)`

This function allows the owner of the contract to set the trusted minter address. There is no test coverage for this function.

Inputs

- `_newMinter`
  - **Control**: Full control.
  - **Constraints**: N/A.
  - **Impact**: N/A.

Branches and code coverage

Intended branches

- The `MinterUpdated` event has been emitted.
  - ☐ Test coverage
- The `minter` address has been updated.
  - ☐ Test coverage

DRAFT

Negative behavior

- The caller is not an owner of the contract.
  - ☐ Negative test

Function: `updateTransferState(TransferState code)`

Owner can block all transfers, whitelist a limited set of addresses, or leave transfers fully open
There are no tests for this function.

Inputs

- `code`
  - **Control**: Full control.
  - **Constraints**: N/A.
  - **Impact**: define if transfer are `FULLY_DISABLED`, `WHITELIST_ENABLED`, or `FULLY_ENABLED`.

Branches and code coverage

Intended branches

- Test to validate all possible transfer cases for `FULLY_ENABLED`; the whitelisted status of accounts should be ignored. All accounts can transfer and receive except the blacklisted.
  - ☐ Test coverage
- Test to validate all possible transfer cases for `WHITELIST_ENABLED`; only whitelisted accounts can transfer or receive, but the owner still can execute the `redistributeLockedAmount` function successfully.
  - ☐ Test coverage
- When the mode is `FULLY_DISABLED`, all transferring actions are restricted even for the owner or minter accounts.
  - ☐ Test coverage

Negative behavior

- The caller is not an owner.
  - ☐ Negative test

Function: `_beforeTokenTransfer(address from, address to, uint256)`

This internal function is executed during minting, burning, and transferring operations. It is intended to validate the status of the caller and `from` and `to` accounts for the different transferring modes: `WHITELIST_ENABLED`, `FULLY_ENABLED`, or `FULLY_DISABLED`.

Branches and code coverage

Intended branches

- The mode is `TransferState.WHITELIST_ENABLED`, `caller == from`, and `from` is whitelisted, cross-chain transferring (`_to = address(0)`).
  - ☐ Test coverage
- The mode is `TransferState.WHITELIST_ENABLED`, the user has allowance, and this user is whitelisted.
  - ☐ Test coverage
- The mode is `TransferState.WHITELIST_ENABLED`, `caller == from`, and `from` is whitelisted.
  - ☐ Test coverage
- The mode is `TransferState.WHITELIST_ENABLED`, `caller == from`, and `from` and `to` are whitelisted.
  - ☐ Test coverage
- The mode is `TransferState.WHITELIST_ENABLED`, `caller == minter`, and the minter is not whitelisted, burning tokens from the non-blacklisted address.
  - ☐ Test coverage
- The mode is `TransferState.WHITELIST_ENABLED`, `caller == minter`, and the minter is not whitelisted, minting tokens to the non-blacklisted address.
  - ☐ Test coverage
- The mode is `TransferState.WHITELIST_ENABLED`, `caller == owner`, and the owner is not whitelisted, burning tokens from the blacklisted address.
  - ☐ Test coverage
- The mode is `TransferState.WHITELIST_ENABLED`, `caller == owner`, and the owner is not whitelisted, minting tokens to the non-blacklisted address.
  - ☐ Test coverage
- The mode is `TransferState.FULLY_ENABLED`, and `caller == minter`, burning tokens from the non-blacklisted address.
  - ☐ Test coverage
- The mode is `TransferState.FULLY_ENABLED`, and `caller == minter`, minting tokens to the non-blacklisted address.

DRAFT

- ☐ Test coverage
- The mode is `TransferState.FULLY_ENABLED`, and `caller == owner`, burning tokens from the blacklisted address.
  - ☐ Test coverage
- The mode is `TransferState.FULLY_ENABLED`, and `caller == owner`, minting tokens to the non-blacklisted address.
  - ☐ Test coverage
- The mode is `TransferState.FULLY_ENABLED`, `caller =! from`, `from` is non-blacklisted, and `to` is non-blacklisted.
  - ☐ Test coverage
- The mode is `TransferState.FULLY_ENABLED`, `caller == from`, `from` is non-blacklisted, and `to` is non-blacklisted.
  - ☐ Test coverage
- The mode is `TransferState.FULLY_ENABLED`, `caller == from`, `from` is not-blacklisted, and the cross-chain transferring is initiated (burning and `to == address(0)`).
  - ☐ Test coverage

Negative behavior

- The mode is `TransferState.WHITELIST_ENABLED`, and the user has allowance, but the user is not whitelisted.
  - ☐ Negative test
- The mode is `TransferState.WHITELIST_ENABLED`, `caller == from`, and `from` is not whitelisted.
  - ☐ Negative test
- The mode is `TransferState.WHITELIST_ENABLED`, `caller == from`, and `from` is whitelisted, but `to` is not whitelisted.
  - ☐ Negative test
- The mode is `TransferState.WHITELIST_ENABLED`, `caller == from`, and `from` is not whitelisted, cross-chain transferring (`_to = address(0)`).
  - ☐ Negative test
- The mode is `TransferState.WHITELIST_ENABLED`, `caller == minter`, and the minter is not whitelisted, trying to transfer from the `minter` account to the whitelisted account.
  - ☐ Negative test
- The mode is `TransferState.WHITELIST_ENABLED`, `caller == minter`, and the minter is not whitelisted, minting tokens to the non-blacklisted address.
  - ☐ Negative test
- The mode is `TransferState.WHITELIST_ENABLED`, `caller == owner`, and the owner is

DRAFT

not whitelisted, burning tokens from the non-blacklisted address.

  - ☐ Negative test
- The mode is `TransferState.WHITELIST_ENABLED`, `caller == owner`, and the owner is not whitelisted, minting tokens to the blacklisted address.
  - ☐ Negative test
- The mode is `TransferState.WHITELIST_ENABLED`, `caller == owner`, and the owner is not whitelisted, transferring tokens from a non whitelisted address to a non whitelisted address.
  - ☐ Negative test
- The mode is `TransferState.WHITELIST_ENABLED`, `caller == owner`, and the owner is not whitelisted, minting tokens to the blacklisted address.
  - ☐ Negative test
- The mode is `TransferState.FULLY_ENABLED`, and `caller == minter`, burning tokens from the blacklisted address.
  - ☐ Negative test
- The mode is `TransferState.FULLY_ENABLED`, and `caller == minter`, minting tokens to the blacklisted address.
  - ☐ Negative test
- The mode is `TransferState.FULLY_ENABLED`, and `caller == minter`, transferring tokens from a previously blacklisted address to another previously blacklisted address.
  - ☐ Negative test
- The mode is `TransferState.FULLY_ENABLED`, and `caller == owner`, burning tokens from the non-blacklisted address.
  - ☐ Negative test
- The mode is `TransferState.FULLY_ENABLED`, and `caller == owner`, minting tokens to the blacklisted address.
  - ☐ Negative test
- The mode is `TransferState.FULLY_ENABLED`, and `caller == owner`, transferring tokens from a previously blacklisted address to another previously blacklisted address.
  - ☐ Negative test
- The mode is `TransferState.FULLY_ENABLED`, `caller =! from`, caller is blacklisted
  - ☐ Negative test
- The mode is `TransferState.FULLY_ENABLED`, `caller == from`, and `from` is blacklisted.
  - ☐ Negative test
- The mode is `TransferState.FULLY_ENABLED`, `caller == from`, and `to` is non-blacklisted.

DRAFT

- ☐ Negative test
- The mode is `TransferState.FULLY_ENABLED`, `caller == from`, `from` is blacklisted, and the cross-chain transferring is initiated (burning and `to == address(0)`).
  - ☐ Test coverage
- The mode is `TransferState.FULLY_DISABLED`; any operation should fail.
  - ☐ Negative test

4.12. Module: iTry.sol

Function: `addBlacklistAddress(address[] users)`

This function allows the `BLACKLIST_MANAGER_ROLE` to grant the `BLACKLISTED_ROLE` to the user. Blacklisted addresses cannot transfer tokens and receive tokens.

There is no test coverage for this function.

Inputs

- `users`
  - **Control**: Full control.
  - **Constraints**: N/A.
  - **Impact**: A blacklisted user cannot transfer or receive iTry tokens.

Branches and code coverage

Intended branches

- If the user is already blacklisted, they will be blacklisted again.
  - ☐ Test coverage
- If the user is already whitelisted, the whitelisted status will be revoked and the user will be blacklisted.
  - ☐ Test coverage
- If the user was not blacklisted/whitelisted before, they will be blacklisted successfully.
  - ☐ Test coverage
- `RoleGranted` has been emitted.
  - ☐ Test coverage
- If the user initially had `WHITELISTED_ROLE`, the `RoleRevoked` will be emitted.
  - ☐ Test coverage

Negative behavior

- The caller is not a `BLACKLIST_MANAGER_ROLE`.
  - ☐ Negative test

Function: `addMinter(address minterContract)`

This function allows the owner of contract to grant the `MINTER_CONTRACT` role for the trusted contract. In the current setup, the iTryIssuer contract has this role. There is no test coverage for this function.

Inputs

- `minterContract`
  - **Control**: Full control.
  - **Constraints**: N/A.
  - **Impact**: The caller with this role can mint new iTry tokens to the non-blacklisted users and burn tokens from the non-blacklisted users. Mostly it will happen during staking and redeeming tokens using the iTryIssuer contract.

Branches and code coverage

Intended branches

- Event `MinterAdded` has been emitted.
  - ☐ Test coverage
- The `MINTER_CONTRACT` has been granted to the `minterContract`.
  - ☐ Test coverage

Negative behavior

- The caller does not have `DEFAULT_ADMIN_ROLE`.
  - ☐ Negative test

Function: `addWhitelistAddress(address[] users)`

This function allows the account with `WHITELIST_MANAGER_ROLE` to grant the `WHITELISTED_ROLE` for the users. If the mode is `TransferState.WHITELIST_ENABLED`, only whitelisted users can transfer and receive tokens. There is no test coverage for this function.

Inputs

- `users`
  - **Control**: Full control.
  - **Constraints**: N/A.
  - **Impact**: List of user addresses that will be granted `WHITELISTED_ROLE`.

Branches and code coverage

Intended branches

- If a user is already whitelisted, they will be whitelisted again.
  - ☐ Test coverage
- If a user is already blacklisted, the blacklisted status will be revoked and the user will be whitelisted.
  - ☐ Test coverage
- If a user was not blacklisted/whitelisted before, they will become whitelisted.
  - ☐ Test coverage
- `RoleGranted` has been emitted.
  - ☐ Test coverage
- If a user initially had `BLACKLISTED_ROLE`, the `RoleRevoked` has been emitted.
  - ☐ Test coverage

Negative behavior

- The caller does not have `WHITELIST_MANAGER_ROLE`.
  - ☐ Negative test

Function: `burnFrom(address account, uint256 amount)`

This function allows burning tokens on behalf of other users using allowance.

Inputs

- `account`
  - **Control**: Full control.
  - **Constraints**: Should provide the allowance for the caller in advance.
  - **Impact**: The account that tokens will be burned from.
- `amount`

DRAFT

- **Control**: Full control.
- **Constraints**: The account should own enough tokens to burn.
- **Impact**: The amount of tokens to burn.

Branches and code coverage

Intended branches

- The mode is `FULLY_ENABLED`, and the caller is blacklisted.
  - ☐ Test coverage
- The mode is `WHITELIST_ENABLED`, the caller is not whitelisted but has the `DEFAULT_ADMIN_ROLE` role, and `from` is blacklisted.
  - ☐ Test coverage
- The mode is `WHITELIST_ENABLED`, the caller is not whitelisted but has the `MINTER_CONTRACT` role, and `from` is non-blacklisted.
  - ☐ Test coverage
- The mode is `FULLY_ENABLED`, the caller has the `DEFAULT_ADMIN_ROLE` role, and `from` is blacklisted.
  - ☐ Test coverage
- The mode is `FULLY_ENABLED`, the caller has the `MINTER_CONTRACT` role, and `from` is non-blacklisted.
  - ☐ Test coverage

Negative behavior

- The caller does not have enough allowance.
  - ☐ Negative test
- The mode is `FULLY_ENABLED`, and the caller is blacklisted.
  - ☐ Negative test
- The mode is `WHITELIST_ENABLED`, and the caller is not whitelisted.
  - ☐ Negative test
- The mode is `FULLY_ENABLED`, and `from` is blacklisted.
  - ☐ Negative test
- The mode is `WHITELIST_ENABLED`, and `from` is not whitelisted.
  - ☐ Negative test
- The mode is `WHITELIST_ENABLED`, the caller is not whitelisted but has the `DEFAULT_ADMIN_ROLE` role, and `from` is not blacklisted.
  - ☐ Negative test

DRAFT

- The mode is `WHITELIST_ENABLED`, the caller is not whitelisted but has the `MINTER_CONTRACT` role, and `from` is blacklisted.
  - ☐ Negative test
- The mode is `FULLY_DISABLED`.
  - ☐ Negative test

Function: `burn(uint256 amount)`

This function allows a user to self-burn tokens.

Inputs

- `amount`
  - **Control**: Full control.
  - **Constraints**: The caller should own enough tokens to burn.
  - **Impact**: The amount of tokens to burn.

Branches and code coverage

Intended branches

- The balance of the caller has been decreased by the `amount`.
  - ☐ Test coverage

Negative behavior

- The caller does not have enough tokens to burn.
  - ☐ Negative test
- The mode is `FULLY_ENABLED`, and the caller is blacklisted.
  - ☐ Negative test
- The mode is `WHITELIST_ENABLED`, and the caller is not whitelisted.
  - ☐ Negative test

Function: `redistributeLockedAmount(address from, address to)`

This function allows the admin to move the full balance of the blacklisted user to the other non-blacklisted account. Firstly, tokens are burned and after that minted again to the non-blacklisted account.

Inputs

- `from`
  - **Control**: Full control.
  - **Constraints**: Should have `BLACKLISTED_ROLE`.
  - **Impact**: Tokens from this account will be burned.
- `to`
  - **Control**: Full control.
  - **Constraints**: Should not have `BLACKLISTED_ROLE`.
  - **Impact**: The full balance of the blacklisted account will be minted for the non-blacklisted account.

Branches and code coverage

Intended branches

- The function is successfully executed when the mode is `TransferState.WHITELIST_ENABLED`.
  - ☐ Test coverage
- The function is successfully executed when the mode is `TransferState.FULLY_ENABLED`.
  - ☐ Test coverage
- The balance of the `from` account is zero.
  - ☐ Test coverage
- The balance of `to` has been increased by the initial `from` balance value.
  - ☐ Test coverage
- The `LockedAmountRedistributed` event has been emitted.
  - ☐ Test coverage

Negative behavior

- The caller is not an admin.
  - ☐ Negative test
- The `from` address does not have the `BLACKLISTED_ROLE`.
  - ☐ Negative test
- The `to` address has the `BLACKLISTED_ROLE`.
  - ☐ Negative test
- The `from` balance is zero.

DRAFT

- ☐ Negative test
- The mode is `TransferState.FULLY_DISABLED`.
  - ☐ Negative test

Function: `removeBlacklistAddress(address[] users)`

This function allows the `BLACKLIST_MANAGER_ROLE` to revoke the `BLACKLISTED_ROLE` from the user.

There is no test coverage for this function.

Inputs

- `users`
  - **Control**: Full control.
  - **Constraints**: N/A.
  - **Impact**: Addresses of the users who will be removed from the blacklist.

Branches and code coverage

Intended branches

- The user address has been blacklisted and has been successfully removed from the blacklist.
  - ☐ Test coverage
- `RoleRevoked` has been emitted.
  - ☐ Test coverage

Negative behavior

- The user address already has the `BLACKLISTED_ROLE`.
  - ☐ Negative test
- The caller does not have the `BLACKLIST_MANAGER_ROLE`.
  - ☐ Negative test

Function: `removeMinter(address minterContract)`

This function allows the owner of the contract to revoke the `MINTER_CONTRACT` role from the contract. There is no test coverage for this function.

Inputs

- `minterContract`
  - **Control**: Full control.
  - **Constraints**: N/A.
  - **Impact**: The address of the account to revoke the `MINTER_CONTRACT` role.

Branches and code coverage

Intended branches

- The `MinterRemoved` event has been emitted.
  - ☐ Test coverage
- The `MINTER_CONTRACT` has been revoked from the `minterContract`.
  - ☐ Test coverage

Negative behavior

- The caller does not have the `DEFAULT_ADMIN_ROLE`.
  - ☐ Negative test

Function: `removeWhitelistAddress(address[] users)`

This function allows the account with `WHITELIST_MANAGER_ROLE` to revoke the `WHITELISTED_ROLE` from the users. If the mode is `TransferState.WHITELIST_ENABLED`, only the whitelisted users can transfer and receive tokens. So the users removed from the whitelist cannot perform the transfer operation after being removed from the whitelist.

There is no test coverage for this function.

Inputs

- `users`
  - **Control**: Full control.
  - **Constraints**: N/A.
  - **Impact**: List of users' addresses that will be removed from the whitelist.

Branches and code coverage

Intended branches

- The user address was previously whitelisted and has been successfully removed from the whitelist.
  - ☐ Test coverage
- The `RoleRevoked` has been emitted.
  - ☐ Test coverage

Negative behavior

- The caller does not have the `WHITELIST_MANAGER_ROLE`.
  - ☐ Negative test

Function: `renounceRole(byte[32], address)`

This function intentionally reverts, preventing anyone, including blacklisted addresses, from renouncing their roles and thereby avoiding access restrictions.

4.13. Module: wiTryOFT.sol

Function: `lzReceive(Origin calldata _origin, bytes32 _guid, bytes calldata _message, address _executor, bytes calldata _extraData)`

This is one of the main functions of the wiTryOFT contract. This function allows receiving messages from the LayerZero endpoint and processing them. Only the trusted and immutable `endpoint` contract can execute this function. Additionally, it performs the validation of the sender address to ensure it is the expected peer for the trusted source chain.

```
struct Origin {
    uint32 srcEid;
    bytes32 sender;
    uint64 nonce;
}
```

The logic of this function is mostly original LayerZero code. Only the `_credit` function is overridden to check if the recipient is blacklisted and to mint tokens to the owner instead.

During receiving the message, on the spoke-chain side, this protocol does not not implement any specific contract to handle the composed message. So if users send the composed message to this chain, they should handle it by themselves.

Inputs

- `_origin`

    - **Control**: N/A.
    - **Constraints**: `srcEid` should be a trusted chain ID, and `sender` should be the expected peer for the `srcEid`.
    - **Impact**: Contains information about the source chain, sender address, and nonce.
  - `_guid`
    - **Control**: Full control.
    - **Constraints**: No constraints.
    - **Impact**: The unique identifier for the received LayerZero message. This `_guid` is used to emit an event and send the composed message to the endpoint.
  - `_message`
    - **Control**: Full control.
    - **Constraints**: No constraints.
    - **Impact**: The payload of the received message. Contains the recipient address, the received amount of tokens, and the composed message.
  - `_executor`
    - **Control**: The address of the executor of the `lzReceive` function in the endpoint contract.
    - **Constraints**: No constraints.
    - **Impact**: Ignored in this contract.
  - `_extraData`
    - **Control**: Full control.
    - **Constraints**: No constraints.
    - **Impact**: Ignored in this contract.

Branches and code coverage

Intended branches

- The compose message has been saved to the endpoint.
  - ☐ Test coverage
- The expected amount of tokens has been received.
  - ☐ Test coverage

Negative behavior

- The recipient is blacklisted.
  - ☐ Negative test
- The recipient is zero address.

DRAFT

- ☐ Negative test
- `srcEid` is not supported.
  - ☐ Negative test
- The sender is not a trusted address for the `srcEid`.
  - ☐ Negative test

Function: `send(SendParam calldata _sendParam, MessagingFee calldata _fee, address _refundAddress)`

This function allows a user to initiate a cross-chain transfer of tokens to another chain. In this case, the wiTryOFT contract is deployed in the spoke chain and is used to transfer tokens to the hub chain or other spoke chains. The caller provides the `SendParam` struct to define the parameters of the cross-chain transfer. The `SendParam` struct is defined as follows:

```
struct SendParam {
    uint32 dstEid; // Destination endpoint ID.
    bytes32 to; // Recipient address.
    uint256 amountLD; // Amount to send in local decimals.
    uint256 minAmountLD; // Minimum amount to send in local decimals.
    bytes extraOptions; // Additional options supplied by the caller to be used
    in the LayerZero message.
    bytes composeMsg; // The composed message for the send() operation.
    bytes oftCmd; // The OFT command to be executed, unused in default OFT
    implementations.
}
```

Additionally, the caller can provide the `composeMsg` data, which will be executed on the destination chain after the tokens are transferred. The recipient should support the `lzCompose` function to handle this `composeMsg`. Otherwise, the `composeMsg` will be saved in the endpoint but cannot be processed by the recipient.

Most of the logic of this function reuses LayerZero code without changes; the only difference is that the `_beforeTokenTransfer` function is overridden to check if the sender of tokens is blacklisted.

Inputs

- `_sendParam`
  - **Control**: Full control.
  - **Constraints**: `dstEid` must be a valid endpoint ID and supported. The caller should own the `amountLD` tokens to send.
  - **Impact**: These parameters contain necessary information to initiate a cross-chain transfer of tokens.

- `_fee`
    - **Control**: Full control.
    - **Constraints**: If the specified fee is not enough and the provided amount of native tokens is different, this function will revert.
    - **Impact**: Specifies the provided fee for the cross-chain transfer.
- `_refundAddress`
    - **Control**: Full control.
    - **Constraints**: No constraints.
    - **Impact**: The address to receive any excess fee.

Branches and code coverage

Intended branches

- The message has been successfully sent to the destination chain.
    - ☐ Test coverage
- The specified amount of tokens have been burned from the caller's balance.
    - ☐ Test coverage

Negative behavior

- The caller is a blacklisted user.
    - ☐ Negative test
- The destination ID is not supported.
    - ☐ Negative test
- Insufficient native fee.
    - ☐ Negative test
- The caller does not own the `amountLD` of tokens to send.
    - ☐ Negative test
- The `to` address is zero.
    - ☐ Negative test

Function: `updateBlackList(address _user, bool _isBlackListed)`

This function allows the blacklisted account or owner to update the blacklist status of a user. The owner of the contract can set the blacklister address using the `setBlackLister` function or reset it to zero address. As a result, only the owner will be able to update the blacklist status of a user.

Inputs

- `_user`
  - **Control**: Full control.
  - **Constraints**: No constraints.
  - **Impact**: The address to update the blacklist status of.
- `_isBlackListed`
  - **Control**: Full control.
  - **Constraints**: No constraints.
  - **Impact**: If true, the user will be added to the blacklist; if false, the user will be removed from the blacklist.

Branches and code coverage

Intended branches

- `_isBlackListed` is true, and the user has been added to the blacklist.
  - ☐ Test coverage
- `_isBlackListed` is false, and the user has been removed from the blacklist.
  - ☐ Test coverage
- The `BlackListUpdated` event has been emitted.
  - ☐ Test coverage

Negative behavior

- The caller is not a blacklister.
  - ☐ Negative test
- The caller is not the owner.
  - ☐ Negative test
- `_isBlackListed` is true, and the user has been blacklisted already.
  - ☐ Negative test
- `_isBlackListed` is false, and the user has been removed from the blacklist already.
  - ☐ Negative test

DRAFT

4.14. Module: wiTryVaultComposer.sol

Function: `handleCompose(address _oftIn, byte[32] _composeFrom, bytes _composeMsg, uint256 _amount)`

This function routes compose callbacks from LayerZero, decodes the compose payload, and then either deposits bridged assets to mint/send shares, starts a cooldown for bridged shares, or performs a fast redeem.

Inputs

- `_oftIn`
  - **Control**: Determined by the OFT contract that initiated compose (the user chooses whether to bridge `ASSET_OFT` or `SHARE_OFT`) from the spoke chain.
  - **Constraints**: Must be either `ASSET_OFT` or `SHARE_OFT`; otherwise, it reverts.
  - **Impact**: Selects the deposit path for assets or the share redemption paths (cooldown or fast redeem).
- `_composeFrom`
  - **Control**: Provided in the compose payload (`composeFrom`) by the remote caller.
  - **Constraints**: N/A.
  - **Impact**: Identifies the depositor/redeemer.
- `_composeMsg`
  - **Control**: Fully controlled by the remote caller composing the message.
  - **Constraints**: Must ABI-decode to `(SendParam,uint256)` — `msg.value` must be superior or equal to `minMsgValue`.
  - **Impact**: `SendParam` drives routing/slippage settings and the chosen share command, while `minMsgValue` gates execution to ensure enough native fee is provided.
- `_amount`
  - **Control**: The amount bridged in the preceding OFT transfer, chosen by the remote user.
  - **Constraints**: N/A.
  - **Impact**: The quantity of assets deposited or shares redeemed / cooled down — sets how many shares are minted or assets returned.

Branches and code coverage

Intended branches

- `_oftIn == ASSET_OFT` — decode payload, validate `msg.value`, deposit `_amount` assets,

and bridge the resulting shares via `_depositAndSend`.

- ☑ Test coverage

- `_oftIn == SHARE_OFT` and `oftCmd == INITIATE_COOLDOWN` — start cooldown for `_composeFrom` using `_amount` shares.
  - ☑ Test coverage
- `_oftIn == SHARE_OFT` and `oftCmd == FAST_REDEEM` — fast redeem `_amount` shares for `_composeFrom` and send assets per `sendParam`.
  - ☑ Test coverage

Negative behavior

- The caller is not `address(this)`.
  - ☑ Negative test
- `msg.value` is below `minMsgValue`.
  - ☑ Negative test
- `_oftIn` is not `ASSET_OFT` or `SHARE_OFT`.
  - ☑ Negative test
- `_oftIn == SHARE_OFT`, but `oftCmd` is unsupported.
  - ☑ Negative test

Function call analysis

- `this._depositAndSend(_composeFrom, _amount, sendParam, address(this)) -> this._deposit(_depositor, _assetAmount) -> this.VAULT.deposit(_assetAmount, address(this))`
  - **What is controllable?** `_assetAmount` comes from the bridged `_amount`, and the vault address is fixed.
  - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
  - **What happens if it reverts, reenters or does other unusual control flow?** N/A.
- `this._depositAndSend(_composeFrom, _amount, sendParam, address(this)) -> this._send(this.SHARE_OFT, _sendParam, _refundAddress) -> SafeERC20.safeTransfer(IERC20(erc20), OFTComposeMsgCodec.bytes32ToAddress(_sendParam.to), _sendParam.amountLD)`
  - **What is controllable?** `_sendParam.to` is user-provided, and `_sendParam.amountLD` is set to the minted share amount.
  - **If the return value is controllable, how is it used and how can it go wrong?** N/A.

    - **What happens if it reverts, reenters or does other unusual control flow?** N/A.
- `this._depositAndSend(_composeFrom, _amount, sendParam, address(this)) -> this._send(this.SHARE_OFT, _sendParam, _refundAddress) -> OFTComposeMsgCodec.bytes32ToAddress(_sendParam.to)`
    - **What is controllable?** N/A.
    - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
    - **What happens if it reverts, reenters or does other unusual control flow?** N/A.
- `this._depositAndSend(_composeFrom, _amount, sendParam, address(this)) -> this._send(this.SHARE_OFT, _sendParam, _refundAddress) -> IOFT(_oft).send{value: msg.value}(_sendParam, MessagingFee(msg.value, 0), _refundAddress)`
    - **What is controllable?** The `_sendParam` fields and `msg.value` originate from the remote caller, and `_refundAddress` is fixed to this contract.
    - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
    - **What happens if it reverts, reenters or does other unusual control flow?** N/A.
- `this._initiateCooldown(_composeFrom, _amount) -> OFTComposeMsgCodec.bytes32ToAddress(_redeemer)`
    - **What is controllable?** N/A.
    - **If the return value is controllable, how is it used and how can it go wrong?** N/A.
    - **What happens if it reverts, reenters or does other unusual control flow?** N/A.
- `this._initiateCooldown(_composeFrom, _amount) -> IStakediTryCrosschain(address(this.VAULT)).cooldownSharesByComposer(_shareAmount, redeemer)`
    - **What is controllable?** `_shareAmount` equals the bridged `_amount`, and `redeemer` comes from `_composeFrom`.
    - **If the return value is controllable, how is it used and how can it go wrong?** Returns the cooled asset amount.
    - **What happens if it reverts, reenters or does other unusual control flow?** N/A.
- `this._fastRedeem(_composeFrom, _amount, sendParam, address(this)) -> OFTComposeMsgCodec.bytes32ToAddress(_redeemer)`
    - **What is controllable?** N/A.
    - **If the return value is controllable, how is it used and how can it go wrong?** N/A.

DRAFT

- **What happens if it reverts, reenters or does other unusual control flow?** N/A.

- `this._fastRedeem(_composeFrom, _amount, sendParam, address(this)) -> IStakediTryCross-chain(address(this.VAULT)).fastRedeemThroughComposer(_shareAmount, redeemer, redeemer)`
    - **What is controllable?** `_shareAmount` is the bridged share amount, and the `redeemer` is user-defined.
    - **If the return value is controllable, how is it used and how can it go wrong?** Returns redeemed assets.
    - **What happens if it reverts, reenters or does other unusual control flow?** N/A.

Function: `_fastRedeem(byte[32] _redeemer, uint256 _shareAmount, SendParam _sendParam, address _refundAddress)`

This function executes a fast redemption for bridged shares. It validates the redeemer address, redeems shares through the vault with fast-redeem fees, updates the send params to match redeemed assets, and forwards assets across chain.

Inputs

- `_redeemer`
    - **Control**: Provided in compose payload by remote caller.
    - **Constraints**: Nonzero address.
    - **Impact**: Recipient of redeemed assets.
- `_shareAmount`
    - **Control**: Bridged share amount from compose flow.
    - **Constraints**: The `shareAmount` must be inferior or equal to `maxRedeem(composer)`.
    - **Impact**: The number of shares redeemed — determines assets to bridge back.
- `_sendParam`
    - **Control**: Set by the remote caller (`dstEid`, `to`, slippage fields, `oftCmd`, etc.).
    - **Constraints**: N/A.
    - **Impact**: Configures the cross-chain asset transfer after redemption.
- `_refundAddress`
    - **Control**: Provided by the caller (`handleCompose` uses `address(this)`).
    - **Constraints**: N/A.

- **Impact**: Destination for any excess `msg.value` from the OFT send.

Branches and code coverage

Intended branches

- Fast-redeem shares through the vault, and revert on zero returned assets.
  - ☑ Test coverage
- Update send parameters and forward redeemed assets via `_send`.
  - ☑ Test coverage

Negative behavior

- `_redeemer` decodes to `address(0)`.
  - ☑ Negative test
- Vault fast-redeem reverts (role, limits, cooldown/feature disabled).
  - ☑ Negative test
- Vault returns zero assets.
  - ☑ Negative test
- `_send` reverts due to invalid send parameters or fee underpayment.
  - ☐ Negative test

Function: `_handleUnstake(Origin _origin, byte[32] _guid, IUnstakeMessenger.UnstakeMessage unstakeMsg)`

This function processes the cross-chain unstake flow with the composer. This is only possible when the cooldown period ends; otherwise, the subcall `unstakeThroughComposer` reverts.

Inputs

- `_origin`
  - **Control**: LayerZero-determined while sending the transaction from the spoke chain.
  - **Constraints**: N/A.
  - **Impact**: Determines the destination chain when sending the unstaked token back on the spoke chain.
- `_guid`
  - **Control**: LayerZero-controlled input.
  - **Constraints**: N/A.

DRAFT

    - **Impact**: Message GUID from LayerZero.
- `unstakeMsg`
    - **Control**: LayerZero-and-spoke-sender partially controlled input.
    - **Constraints**: N/A.
    - **Impact**: Message data, used to determine `msgType` and `unstakeMsg`.

Branches and code coverage

Intended branches

- Sanity check on function inputs (`user` unstaking and `sourceEid`).
    - ☑ Test coverage
- Send the cross-chain `ASSET_OFT`s back on the spoke chain, via `_send`.
    - ☐ Test coverage

Function: `_initiateCooldown(byte[32] _redeemer, uint256 _shareAmount)`

This function converts the redeemer identifier to an address and instructs the staking vault to burn the bridged shares and start a cooldown for the redeemer.

Inputs

- `_redeemer`
    - **Control**: Supplied by the remote caller in the compose message.
    - **Constraints**: Must decode to a nonzero address.
    - **Impact**: Determines which address is used for the cooldown.
- `_shareAmount`
    - **Control**: The share amount bridged in the compose flow.
    - **Constraints**: `shares` must be inferior to `maxRedeem(composer)`.
    - **Impact**: The number of shares burned — determines the cooled asset amount returned for the redeemer.

Branches and code coverage

Intended branches

- Reject zero redeemer address.
    - ☑ Test coverage
- Call vault to start cooldown.

DRAFT

- [x] Test coverage

Function: `_lzReceive(Origin _origin, byte[32] _guid, bytes _message, address _executor, bytes _extraData)`

This is the function used when a cross-chain unstake is executed from a spoke chain. This is only possible when the cooldown period ends; otherwise, the subcall `unstakeThroughComposer` reverts.

Inputs

- `_origin`
  - **Control**: LayerZero-determined while sending the transaction from the spoke chain.
  - **Constraints**: N/A.
  - **Impact**: Determines the destination chain when sending the unstaked token back on the spoke chain.
- `_guid`
  - **Control**: LayerZero-controlled input.
  - **Constraints**: N/A.
  - **Impact**: Message GUID from LayerZero.
- `_message`
  - **Control**: LayerZero-and-spoke-sender partially controlled input.
  - **Constraints**: N/A.
  - **Impact**: Message data, used to determine `msgType` and `unstakeMsg`.
- `_executor`
  - **Control**: N/A.
  - **Constraints**: N/A.
  - **Impact**: Not used
- `_extraData`
  - **Control**: N/A.
  - **Constraints**: N/A.
  - **Impact**: Not used

Branches and code coverage

Intended branches

- Decode LayerZero `_message`, if `msgType` is equal to `MSG_TYPE_UNSTAKE`, and process the cross-chain unstaking; otherwise, revert.

DRAFT

- ☑ Test coverage

Function: `_refund(address _oft, bytes _message, uint256 _amount, address _refundAddress)`

This function reconstructs a `SendParam` from the failed compose message, forces a `TYPE_3` options header, and resends the bridged amount back to the source chain/sender using the specified OFT.

Inputs

- `_oft`
  - **Control**: Passed in from the `lzCompose` context (the OFT that originally bridged funds).
  - **Constraints**: Expected to be `ASSET_OFT` or `SHARE_OFT` validated upstream.
  - **Impact**: Selects which token is refunded.
- `_message`
  - **Control**: Set by the LayerZero compose flow and contains `srcEid` / `composeFrom` and amount metadata.
  - **Constraints**: Must decode via `OFTComposeMsgCodec`.
  - **Impact**: Drives refund destination and destination chain.
- `_amount`
  - **Control**: The amount that failed to process, forwarded as refund `amountLD`.
  - **Constraints**: Must fit token balance held for refund.
  - **Impact**: Quantity refunded to the original sender.
- `_refundAddress`
  - **Control**: Provided by the caller.
  - **Constraints**: Receives any excess native fee from the OFT refund send.
  - **Impact**: Where leftover `msg.value` is returned.

Branches and code coverage

Intended branches

- Build refund `SendParam` (`dstEid` / `from`, `to` / `composeFrom`, `amountLD`, `TYPE_3` options), and invoke OFT send.
  - ☑ Test coverage

Negative behavior

- OFT send reverts (e.g., insufficient `msg.value` or invalid params).

- ☐ Negative test

DRAFT

5. Assessment Results

During our assessment on the scoped iTRY contracts, we discovered six findings. No critical issues were found. Four findings were of medium impact and two were of low impact.

5.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.

DRAFT